TECHNISCHE UNIVERSITÄT DRESDEN

Faculty of Computer Science Institute of Software- and Multimedia-Technology

Chair of Software Technology

Graphical Management of Multiple Runtime Models to Improve the Comprehension of Contextual Behavioral Programs

Nick Ruider

Born on: 27th September 2001 in Dresden
Matriculation number: 4935137

Master Thesis

to achieve the academic degree

Master of Science (M.Sc.)

First referee
Dr.-Ing. Sebastian Götz
Second referee
Prof. Dr. rer. nat. habil. Uwe Aßmann
Supervisor
Dr.-Ing. Sebastian Götz

Submitted on: Dresden, 5th June 2026

Statement of authorship

I hereby certify that I have authored this document entitled *Graphical Management of Multiple Runtime Models to Improve the Comprehension of Contextual Behavioral Programs* independently and without undue assistance from third parties. No other than the resources and references indicated in this document have been used. I have marked both literal and accordingly adopted quotations as such. There were no additional persons involved in the intellectual preparation of the present document. I am aware that violations of this declaration may lead to subsequent withdrawal of the academic degree.This document was written without generative AI. The tool *Grammarly* was only used to improve spelling and grammar.

Dresden, 5th June 2026

Nick Ruider

Acknowledgments

I want to express my sincere gratitude to my supervisor, Dr.-Ing. Sebastian Götz, for his invaluable suggestions and constructive feedback throughout the thesis process. His guidance was crucial in sharpening the research focus and greatly improving the tool's core features.

I would also like to thank Tom Felber for his patient explanations of the FMBP framework and the underlying UVL extension. His support was indispensable, as without it, the integration of *models@run.time* via FMBP would not have been achievable.

Finally, I am deeply grateful to my friends and family for their constant support, patience, and encouragement. I want to extend special thanks to Thomas Nürk for his kindness, thorough proofreading, and engaging technical discussions. Their support made the work on this thesis not only more manageable, but also a genuinely rewarding experience.

TECHNISCHE UNIVERSITÄT DRESDEN

Faculty of Computer Science Institute of Software- and Multimedia-Technology

Chair of Software Technology

Abstract

Behavioral Programming (BP) is a scenario-based paradigm in which programs become increasingly difficult to comprehend at scale, yet existing tooling lacks intuitive graphical support. We present two Visual Studio Code extensions built on the Eclipse Graphical Language Server Platform (GLSP) that address this gap through graphical management of multiple runtime models for contextual behavioral programs.

The first extension provides a lightweight graphical editor for feature models expressed in the Universal Variability Language (UVL). The second enriches UVL with BP-specific constructs. It introduces two new root features, *Env* and *Config*, to capture the contextual aspects of a behavioral program. To represent BP concepts within the feature model, we model *b-threads* as a dedicated feature type, while *b-events* are expressed as feature attributes directly embedded in the tree structure. The extension further supports the BP-specific constraints required by this enriched model defined by Felber and Götz.

For runtime monitoring, we adopt the *models@run.time* paradigm by extending Felber's FMBP framework. During execution, FMBP emits server-sent events for each selected *b-event*, which the GLSP server reflects in the editor via live highlighting. Dynamic environment attributes are similarly transmitted and rendered on the client side. Crucially, the toolchain supports simultaneous visualization of multiple runtime models, enabling several independent behavioral programs to be observed and compared within a single graphical environment.

We demonstrate the tools with a water-tank scenario and a multi-drone example featuring four concurrent runtime models. Both extensions are published on *OpenVSX* with source code available on GitHub.

Contents

List of Abbreviations

ANTLR ANother Tool for Language Recognition. 39–41, 52, 53

BP Behavioral Programming. 9–14, 23, 25, 29–31, 38–42, 45–47, 49–52, 54, 55, 60, 61, 70, 71, 77, 78, 80–84

COBP Context-Oriented Behavioral Programming. 13

EMF Eclipse Modeling Framework. 21, 32, 36, 37

FODA Feature-Oriented Domain Analysis. 15, 16

GLSP Graphical Language Server Platform. 10, 11, 19–21, 31–45, 52, 56–61, 63–65, 67, 70–72, 75, 76, 78, 80–84

IDE Integrated Development Environment. 20, 25, 27, 31, 37, 38

LSP Language Server Protocol. 20, 25, 26

SSE Server-Sent Events. 43, 44, 50, 51, 62–64, 66, 67, 70–72, 78, 80, 84

UML Unified Modeling Language. 17, 33, 47

UVL Universal Variability Language. 10, 11, 18, 23–27, 30, 35–42, 44–46, 49, 52, 55–57, 59–61, 68–71, 75–78, 80–84

UVLS UVL Language Server. 25, 26, 31, 39, 44

1 Introduction

1.1 Motivation

Specifying the requirements of a software system is one of the most challenging tasks in software engineering. Despite decades of research, capturing the intended behavior of a system in a complete, consistent, and comprehensible manner remains an open problem. One well-established strategy to address this challenge is the use of use cases and scenarios to express requirements, allowing stakeholders to describe system behavior from a user-centric perspective. Behavioral Programming (BP) builds directly on this idea by elevating scenarios from mere documentation artifacts to first-class implementation constructs. In BP, a program is composed of individual behavioral threads (*b-threads*), each encoding a scenario that specifies which events to request, which to wait for, and which to block. Together, these *b-threads* execute concurrently and coordinate, employing a shared event mechanism to produce the overall system behavior [1].

While this paradigm offers notable advantages, it introduces a serious challenge as programs grow in size and complexity. The interplay between *b-threads* and the events they select becomes increasingly difficult to reason about, both at the modeling stage before any code is executed and at runtime when the system is live. Understanding how multiple *b-threads* interact with one another, and how their collective behavior emerges from their individual specifications, quickly exceeds the cognitive capacity of a developer relying on source code alone.

Prior work by Felber and Götz has begun to address these challenges. They introduced a novel runtime debugger for behavioral programs that collects BP-specific execution information and presents it via a web-based frontend, helping developers understand program behavior at runtime and identify bugs [2]. However, the high-level architecture of a behavioral program remains difficult to grasp through this lens alone. To address the modeling dimension of the problem, Felber and Götz further proposed an Architecture Description Language (ADL) for

context-aware behavioral programs, grounded in the Universal Variability Language (UVL) [3]. UVL is a formalism commonly used for feature modeling in the field of Software Product Lines. Their approach augments an existing UVL language server with BP-specific semantics and introduces a Python backend called FMBP [4]. Together, these components allow developers to model a behavioral program using UVL and to enforce dynamic changes to the feature model at runtime, bridging the gap between static architectural modeling and dynamic program execution. Yet, both the runtime debugger and the UVL-based modeling approach lack a graphical representation that would allow developers to intuitively inspect and interact with the architectural model of a behavioral program.

1.2 Goals of the thesis

In this thesis, we aim to improve the comprehension of contextual behavioral programs by introducing a graphical management layer. Building on UVL with BP semantics contributed by Felber and Götz, we propose an interactive graphical editor that allows developers to visualize and manipulate the feature model of a behavioral program at runtime. We use the Graphical Language Server Platform (GLSP) as the implementation framework, as it provides a proven foundation for building language-specific graphical editors that integrate into various development environments [5]

The UVL feature model serves as the central artifact. The graphical editor should be able to project the UVL model as in other feature modeling tools, such as FeatureIDE [6]. Although the focus is on the comprehension of behavioral programs, we should design the editor with extensibility in mind, separating the core graphical representation of base UVL models from the specific BP semantics.

A major concern of this thesis is the *models@run.time* approach [7]. Changes made graphically by the developer must be translated into valid modifications of the live behavioral program, while changes arising from program execution must be automatically reflected in the visual representation. Additionally, the proposed editor should support multiple runtime instances, enabling simultaneous visualization and interaction with multiple behavioral programs within a single graphical interface.

The following research questions frame the thesis.

RQ1 How feasible is the graphical visualization of UVL feature models for improving the comprehension of contextual BP?

RQ2 How can BP concepts be integrated into a graphical tool while maintaining extensibility for future features and analyses?

RQ3.1 How can changes to the feature model during the execution of a behavioral program be automatically reflected in the graphical representation?

RQ3.2 How can user interactions with the graphical model be translated into valid runtime modifications of the underlying behavioral program?

The remainder of this thesis is structured as follows. Chapter 2 establishes the theoretical foundations underlying this work, covering feature modeling, the principles of BP, and the GLSP framework as the technical basis for the graphical editor. Chapter 3 surveys the state of the art in those domains, identifying existing tools and approaches that relate to the goals of this thesis and highlighting the research gap that this work aims to fill. Chapter 4 presents the conceptual design of the proposed graphical editor, detailing the mapping from UVL constructs to graphical notation and the architectural decisions that govern the bidirectional relationship between the diagram and the live model. Chapter 5 describes the realization of the design, covering the integration points between the graphical editor and the underlying extension to FMBP. Chapter 6 applies the tool to three representative examples. Finally, Chapter 7 summarizes the main contributions of this thesis and outlines directions for future work.

2 Background

This chapter establishes the theoretical foundations of this thesis. It introduces behavioral programming and feature modeling as the core conceptual pillars. Building on this, it explains *models@run.time* as the perspective on runtime adaptation and presents the Graphical Language Server Platform as the technological basis for graphical model tooling. It concludes with the visual representation theory, which grounds the design rationale for cognitively effective diagram notations.

2.1 Behavioral Programming

Behavioral Programming (BP) is a paradigm for programming reactive systems. It bridges the gap between requirements and implementation by expressing system behavior as executable scenarios and use cases. BP is based on independent scenarios that correspond to individual system requirements [1]. Given that each scenario acts as a standalone module, developers can incrementally extend and refine the system by adding new behaviors, rules, or tactics without modifying existing code. This development style makes BP well suited to complex, reactive systems [8].

2.1.1 Foundations of Behavioral Programming

The primary building blocks of BP are behavioral threads (b-threads). Each b-thread runs concurrently and captures one specific scenario that the system should or should not follow. The b-threads are decoupled and typically do not know of one another [1].

B-threads communicate through an enhanced publish/subscribe protocol using the `bSync` method. When a b-thread reaches a synchronization point, it invokes `bSync` and suspends

execution until the arbiter evaluates the joint state of all b-threads [1]. At each synchronization point, every b-thread submits three sets of events to the event arbiter:

- **requested:** The b-thread proposes these events for execution.
- **waited-for:** These events are not triggered by the b-thread, but it asks to be notified if another thread successfully requests them.
- **blocked:** The b-thread explicitly forbids these events from occurring, acting as a veto against other b-threads' requests.

An event is triggered only if at least one b-thread requests it and none block it [8].

2.1.2 Context-Oriented Behavioral Programming

Context-Oriented Behavioral Programming (COBP), formalized by Elyasaf [8], extends BP by integrating dynamic environmental data and system state into the behavioral model. In Harel's original BP, systems are built from independent b-threads that communicate only by requesting, waiting for, and blocking events at synchronization points [1]. While this keeps threads isolated, it leaves no explicit, first-class mechanism for representing or changing context. As a result, sharing contextual data among b-threads requires structural workarounds [8].

Elyasaf addresses this in COBP by introducing formal extensions that natively handle context. Instead of standard b-threads, COBP uses Context-Aware Behavioral Threads (CBTs), each bound to a query over the system's contextual data. When a CBT's query yields a new result, the system spawns a live copy of that CBT, passing the query result as a local variable called the seed, allowing the thread to execute its logic in that specific context. COBP also connects behavioral events to data updates through an effect function, which translates selected events into updates to the contextual data structure before propagating the updated context to all live copies [8].

An empirical study suggests that this explicit context modeling helps developers comprehend complex, context-dependent guard rules [9]. It is also well suited for scalable systems that must adapt to changing conditions, such as smart devices and robotic controllers. At the same time, COBP introduces a steeper learning curve and higher cognitive load, which can make systems more difficult to comprehend and align with requirements than classical BP does.

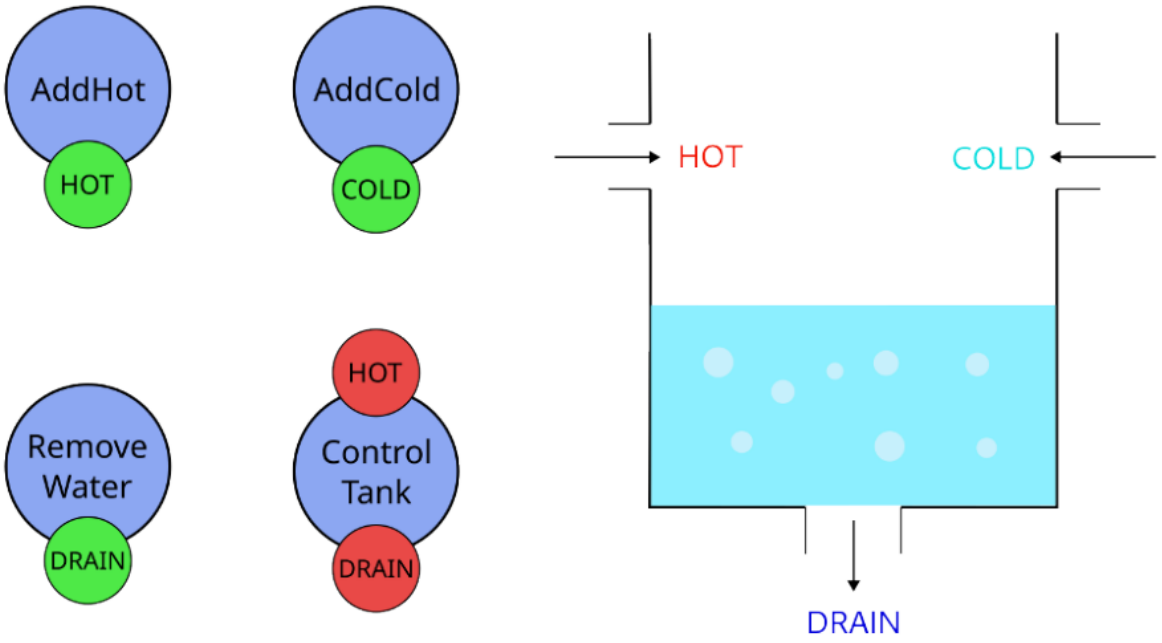


Figure 2.1: Water Tank Example; Water is too hot and too little, so *HOT* and *DRAIN* are blocked. Taken from [3].

2.1.3 Water Tank Example

To illustrate BP in a dynamic environment, Harel introduces the water tank scenario [1, 8]. The system maintains target water levels and temperatures by controlling a hot-water tap, a cold-water tap. Felber and Götz extend this scenario with an additional drain [3]. Instead of a monolithic control structure, independent concurrent b-threads are used, each managing a single requirement. Figure 2.1 illustrates one representative runtime situation of this example.

- **AddHot**: Requests the *HOT* event (adds hot water).
- **AddCold**: Requests the *COLD* event (adds cold water).
- **RemoveWater**: Requests the *DRAIN* event (decreases water level).
- **ControlTank**: Acts as a supervisor that enforces operational limits by blocking specific events.

During execution, the b-threads synchronize and vote on the next action. An event is triggered only if it is requested and not vetoed by a supervising b-thread. As shown in Figure 2.1, *HOT* and *DRAIN* are blocked in this depicted state because the water is too hot and the water level is too low. This decoupled architecture allows developers to add new rules, such as a b-thread that prevents overflow, without altering the existing b-threads [1, 3].

2.2 Feature Modeling

Feature modeling provides a structured way to describe commonalities and variabilities within a family of related systems. It is central to this thesis because the proposed graphical extensions build on feature models to represent both static product structure and runtime-relevant variation. The following subsections introduce the background of Software Product Lines, explain how feature trees encode relationships between features, and outline how the Universal Variability Language captures these ideas in a textual syntax.

2.2.1 Software Product Lines

Software Product Lines (SPL) shift development from isolated systems to a reuse-oriented engineering approach [10]. The core idea is to build a shared asset base that can be systematically reconfigured into a family of related products tailored to specific market needs [11]. By managing commonality and variability across the portfolio, organizations can reduce development costs and shorten time-to-market [12]. The process is typically divided into domain engineering and application engineering. During domain engineering, the focus lies on defining the relevant features and constraints that characterize the product family. Application engineering then uses this model to derive specific products tailored to individual stakeholder requirements [13, 14].

Dynamic Software Product Lines (DSPL) extend these traditional SPL concepts by integrating both static variability in space and dynamic variability over time into a unified framework [15]. Rather than completely shifting variability resolution away from design time, DSPL support multiple binding times to handle features across different operational modes. This approach allows developers to clearly distinguish between static features finalized prior to product derivation and dynamic features left open as variation points. These enable autonomous system adaptation and reconfiguration at runtime [16].

Feature models in general define the boundaries of what can be built and provide a structured view of product variation, making them a natural tool for managing this variability.

2.2.2 Feature Trees

Feature-Oriented Domain Analysis (FODA) establishes the foundation for hierarchical decomposition by defining features as user-visible characteristics identified through rigorous domain analysis [17]. A feature model consists of a hierarchically arranged collection of features. Its structural integrity relies on the relationships between parent and child features [17, 18].

Across the literature [11, 18, 19, 20, 21], basic feature models contain the following relationship types:

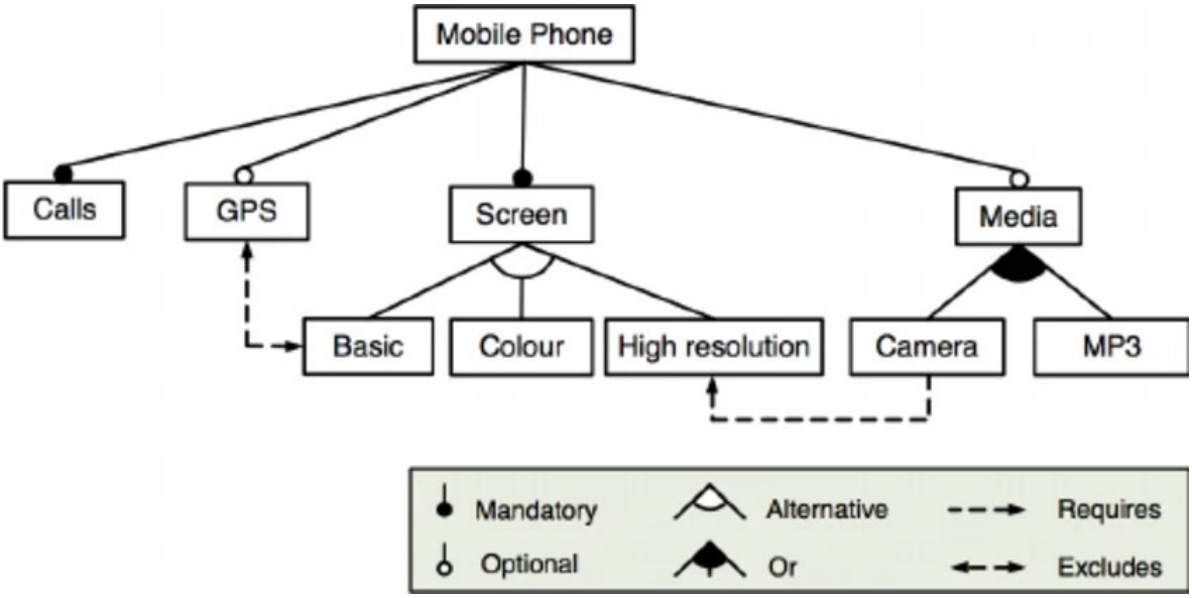


Figure 2.2: Example of a feature tree for a mobile phone, including cross-tree constraints. Taken from [21].

- **Mandatory.** A child feature has a *mandatory* relationship with its parent feature if the child is part of all products in which the parent is included.
- **Optional.** A child feature has an *optional* relationship with its parent feature if the child is optionally part of all products in which the parent is included.
- **Or.** A set of child features has an *or* relationship with their parent feature when at least one child is active in products where the parent is included.
- **Alternative.** A set of child features has an *alternative* relationship with their parent feature when exactly one child is active in products that include the parent. In other literature [11, 20], the *alternative* relation can be combined with *mandatory* or *optional*, which allows for a selection of zero elements as well.

Beyond these parent-child relationships, feature models may contain cross-tree constraints between features. Typical examples are *requires* and *excludes*. A *requires* constraint enforces that selecting one feature also requires selecting another. An *excludes* constraint stipulates that two features cannot be included in the same configuration. More expressive formulations are also possible, including propositional constraints such as `A implies not B or not C` [22]. Figure 2.2 illustrates a feature tree with such cross-tree constraints.

2.2.3 Advancements to Feature Trees

Advanced feature modeling extends the traditional binary logic of FODA by introducing quantitative constraints and metadata [21]. These extensions support more complex configurations

and more detailed resource tracking.

Cardinality-Based Modeling

This approach borrows from Unified Modeling Language (UML) multiplicities to define how often and how many instances of a feature may appear in a configuration [18, 23].

Feature cardinality. A feature cardinality is assigned an interval $[n \dots m]$, where n is the minimum and m is the maximum number of instances permitted. This generalizes traditional relationships:

- Mandatory: $[1 \dots 1]$
- Optional: $[0 \dots 1]$
- Clones: Any range where $m > 1$, allowing multiple distinct instances of the same feature.

Group cardinality. This restricts the selection of child features when a parent is active. Represented by the interval $[n \dots n]$, it dictates that at least n and at most m children must be chosen. This generalizes traditional relationships:

- Alternative: $[1 \dots 1]$
- Or-Relationship: $[1 \dots k]$, where k is the total number of available sub-features.

Attribute-Based Modeling

Attribute-based modeling extends standard feature models by attaching additional information to features as attributes. These extended models are often referred to as extended, advanced, or attribute-based feature models [21]. When the basic tree structure is not expressive enough, attributes provide a way to capture richer feature-specific information and support more advanced analyses.

Attributes are typically represented as key-value pairs and are commonly described by at least a name, a type, a domain, and a value. Depending on the model, an attribute may store a numeric value, a boolean flag, or a string [21, 24].

Attributes are useful as metadata containers for tool-specific information or for values that should remain directly attached to a feature in the model structure. They also enable metric computation, such as calculating the total price of a selected configuration, by aggregating attribute values across all chosen features [21]. In addition, attribute-based models support constraints over numeric and textual domains, enabling rules such as budget limits or consistency checks on string values [24]. This makes attributes a practical extension when propositional feature constraints alone are insufficient.

2.2.4 Universal Variability Language

The Universal Variability Language (UVL) emerged to standardize these concepts into a unified textual exchange format, reducing fragmentation across modeling tools. Designed for both human readability and automated processing, UVL uses indentation-based nesting to represent the tree hierarchy and provides a dedicated block for cross-tree constraints [6, 25].

To balance simplicity with expressiveness, UVL features an extensible design structured into three major language levels: Boolean, Arithmetic, and Type [25]. The Boolean level serves as the core language and can be easily encoded as a SAT problem. The Arithmetic and Type levels introduce numeric constraints and typed features that require more complex reasoning engines, such as SMT solvers. Additionally, UVL includes optional minor levels that support advanced constructs such as feature and group cardinalities, as well as string constraints [25].

To address the complexity of large-scale industrial product lines, UVL incorporates an import mechanism that decomposes massive feature models into smaller, manageable sub-models [25]. These sub-models can be maintained independently and composed into an overall model, using a namespace syntax similar to Java imports. Because of its standardized, extensible architecture, UVL has been successfully integrated as a pivot language in various variability modeling and transformation frameworks, such as FeatureIDE, flamapy, and TraVarT [6].

```
features
   "Mobile Phone"
      mandatory
         Calls
         Screen
            alternative
               Basic
               Colour
               "High resolution"
      optional
         GPS
         Media
            or
               Camera
               MP3
constraints
   Camera => "High resolution" // requires
   !(GPS & Basic) // excludes
```

Listing 2.1: UVL specification of the mobile phone feature model shown in Figure 2.2

In this thesis, UVL serves as the foundational abstract syntax on which the proposed graphical extensions are mapped. Listing 2.1 shows the UVL representation of the same mobile phone

example introduced in Figure 2.2. It is a direct textual encoding of that feature tree, including its hierarchy and cross-tree constraints.

2.3 Models@run.time

Models@run.time extend the principles of Model-Driven Engineering from the traditional development phase into the active execution environment of a software system. By definition, they are causally connected self-representations of a software system that emphasize its structure, behavior, or goals from a high-level problem-space perspective [7, 26]. Traditional models are commonly used for documentation, communication, and initial code generation prior to deployment. In contrast, *models@run.time* act as active abstractions during system execution. They enable developers or the system itself to monitor, reason about, and reconfigure the software without interrupting execution [7].

The fundamental mechanism that enables this continuous execution support is computational reflection, which establishes a bidirectional causal connection between the runtime model and the underlying software system. This link keeps the internal representation aligned with the domain. As a result, changes in the system's state or behavior are automatically captured and reflected in the runtime model. Conversely, adaptations performed on the model at the problem-space level are propagated down to the executing system [26].

This causal relationship is an important prerequisite for building dynamic, self-adaptive, and long-term operating software systems. It ensures that adaptation engines and human operators receive an accurate, up-to-date view of the system's state. Furthermore, it allows safe adaptations to be enacted at the high level of the problem-space model, mitigating the risks associated with direct, low-level manipulations of executing code [7, 26].

Despite its advantages, maintaining a continuous causal connection introduces substantial research challenges, particularly when applied to fine-grained, code-level artifacts such as abstract syntax trees [27]. First, traditional compiler artifacts are usually discarded after compilation. They must therefore be kept "alive" and enriched with runtime-specific knowledge. Second, novel synchronization techniques are required to handle temporary synchronization lags and race conditions. The latter occurs when multiple runtime models attempt to control the same underlying system concurrently. Additionally, transaction-safe connections are needed to allow safe rollbacks upon failure [26].

2.4 Graphical Language Server Platform

The Graphical Language Server Platform (GLSP) transfers the client-server idea of language tooling to graphical modeling and provides a basis for modern diagram editors.

2.4.1 Language Server Protocol

Introduced by Microsoft, RedHat, and Codenvy in 2016, the Language Server Protocol (LSP) has become the de facto standard for language editing support in modern Integrated Development Environments (IDEs) [28]. Its core idea is to split a monolithic IDE architecture into two decoupled components: a *language client* and a *language server*. The client provides the user-facing editor and integration, while the server encapsulates language services such as parsing, code completion, diagnostics, and go-to-definition. The two components communicate via JSON-RPC, a lightweight protocol for stateless remote procedure calls [28, 5]. A single language server can be reused across any LSP-compliant editor, such as VS Code or Eclipse Theia [28]. This decoupling reduces the traditional $m \times n$ integration problem to $m + n$. Given LSP's success in the textual domain, the same architectural principle has since been applied to graphical modeling [5, 29].

2.4.2 Extending LSP to Graphical Modeling

The Graphical Language Server Platform is an open-source framework hosted by the Eclipse Foundation that applies the client-server architecture of LSP to diagram editors [5]. While LSP operates on text documents and cursor positions, GLSP addresses the specific demands of graphical models, including elements that occupy geometric space, nested hierarchies, and constrained connections between nodes. GLSP likewise communicates via an extensible JSON-RPC protocol and separates concerns into two components [5]:

- GLSP Server: Handles model management, editing logic, validation, and transformation of the underlying semantic source models.
- GLSP Client: Renders the visual diagram (typically as SVG in the browser) and manages user interactions such as panning, zooming, and editing.

In the GLSP, *Actions* are the fundamental message objects exchanged between the client and server to facilitate all communication. They represent specific events, requests, or commands, such as a user moving a node or the server pushing an update, that trigger state changes and keep the diagram synchronized across the system [5].

To support tool developers, the framework provides *server frameworks* for state management and layout computation [30, 31], as well as a *client framework* built on HTML5 and SVG [32]. Additionally, *platform integrations* allow the same diagram client to be deployed as a standalone web application [32] or embedded into VS Code [33] and Eclipse Theia [34]. Figure 2.3 gives a compact overview of the main components and how they interact.

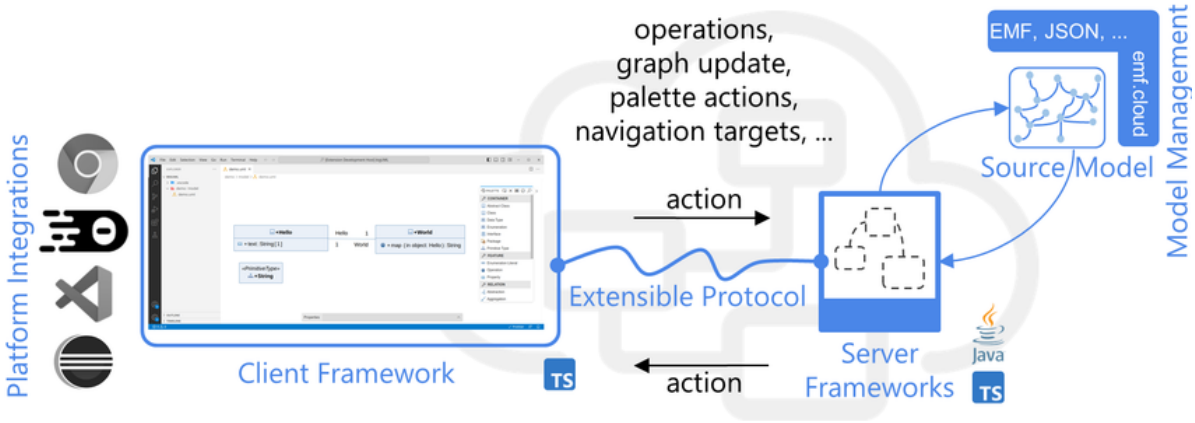


Figure 2.3: Overview of GLSP components and their interplay. Taken from [5].

2.4.3 The Graphical Model Transformation Workflow

The architecture of GLSP rests on a two-model abstraction. The *Source Model* serves as the persistent source of truth and may itself be split into separate domain and notation concerns. For example, the Eclipse Modeling Framework (EMF) can manage semantic logic independently of visual metadata such as node positions and element dimensions [35]. From the source model, the server derives the *Graphical Model* (GModel), a typed and attributed graph that describes the diagram's structure and geometry. It serves as the transient artifact exchanged with the client [5].

The natural interaction flow commences with the client encapsulating a user action and sending it to the server for validation and execution rather than applying it directly as a model edit [5]. The server processes the request, potentially modifies the source model, transforms the updated state into a new GModel, and returns it to the client. The incoming GModel is then rendered as SVG. This division also reflects a clear split in authority. The server determines which elements are rendered based on domain state, while the client controls how they appear through CSS and SVG styling. For trivial operations such as dragging, the client may apply immediate visual feedback optimistically, with final state reconciliation reserved for the server.

A practical advantage of this architecture is cross-runtime interoperability. A Java-based server can communicate seamlessly with a TypeScript client, allowing complex semantic logic to run on the Java Virtual Machine while the user interface remains a responsive web application [35, 36].

2.5 Visual Representation Theory

The choice between textual and graphical representation has measurable consequences for human comprehension and problem-solving efficiency. Textual representations are inherently one-dimensional and processed serially by the auditory-linguistic system [37]. This requires the reader to hold multiple, potentially distant elements in working memory while searching linearly through the representation. By contrast, graphical representations are two-dimensional and processed in parallel by the visual system to which approximately a quarter of the brain's total capacity is devoted [38].

The concept of *computational offloading* [37] formalizes this distinction. Diagrams index information by spatial location rather than sequential position, allowing elements that must be reasoned about together to be grouped physically. Spatial, topological, and geometric relationships can thereby be perceived directly, whereas equivalent inferences drawn from text require explicit and effortful computation. This reduces extraneous cognitive load as identified by Cognitive Load Theory and frees working memory for substantive reasoning tasks [39].

Despite the well-established advantages of graphical representation, the design of visual notations in software engineering has historically been treated as secondary to semantic concerns. Often it is driven by intuition rather than principled theory [38]. The *Physics of Notations* addresses this deficit by establishing a prescriptive, empirically grounded framework for designing cognitively effective visual languages [38]. The theory derives nine principles from cognitive science and visual perception research [38]:

- Semiotic Clarity: Each semantic construct should map to one distinct graphical symbol; violations produce redundancy, overload, excess, or deficit.
- Perceptual Discriminability: Symbols must be easy to tell apart; discriminability depends on the "visual distance" across variables such as shape, color, and size.
- Semantic Transparency: Symbols should visually suggest their meaning so that appearance provides intuitive cues and reduces learning effort.
- Complexity Management: Notations should offer mechanisms (for example, modularization, hierarchical grouping, or layered views) to present information incrementally and avoid cognitive overload.
- Cognitive Integration: Provide contextual anchors, overview maps, and cross-references so readers can relate individual diagrams to the larger system of representations.
- Visual Expressiveness: Leverage multiple visual channels (color, size, brightness, texture, position, etc.) so the notation can encode information across a richer perceptual space.

- **Dual Coding:** Combine concise textual labels or annotations with graphics to leverage both verbal and visual cognitive systems and improve comprehension and retention.
- **Graphic Economy:** Keep the visual vocabulary compact so distinct symbols remain within human discriminative abilities, especially for novice users.
- **Cognitive Fit:** Tailor the notation to task and audience, offering simplified and expert variants so the representation matches users' goals and skills.

These principles interact in practice: increasing Visual Expressiveness (e.g., adding color) can improve Perceptual Discriminability and support offset problems introduced by a large graphic vocabulary [38].

2.6 Drones Example

In the previous sections, we used the mobile phone UVL model and the water tank BP model as examples. The drones example, introduced in FMBP [4], serves as a core evaluation use case for the thesis. It requires interacting with multiple concurrent runtime models, represented by several instances of behavioral programs running simultaneously, which is a key aspect of the thesis.

```
features
   Drone
      alternative
         Patrol {type 'BThread', PATROL {type 'BEvent', requested true}}
         Follow {type 'BThread', FOLLOW {type 'BEvent', requested true}}
      optional
         Charge {type 'BThread', CHARGE {type 'BEvent', requested true, priority
             1}}
   Env {type 'Env', charge 100, is_charging 0}
   Config {type 'Config', patrol_targets '0,1,2,3,4', is_leader 1,
       leader_to_follow '1', follow_distance 50}

constraints
   (Env.charge < 20 | Env.is_charging == 1) & Env.charge < 100 <=> requested(
       CHARGE)
   requested(PATROL) <=> Config.is_leader == 1
```

Listing 2.2: UVL specification of a single drone's feature model. Taken from [40].

The scenario models a small surveillance deployment consisting of four autonomous drones, whose spatial movements and interactions are visualized in real time using an *Alchemist* simulation environment [41]. The swarm's logic is governed entirely by behavioral programming. Each drone possesses an identical individual feature model represented in UVL, which allows capabilities, assignments, and operational parameters to be defined within the model

and dynamically applied to each drone at runtime. Listing 2.2 shows an example of the UVL model for a single drone. Role assignment is managed through dedicated *Patrol* and *Follow* b-threads, which enforce mutually exclusive behavioral modes. Drones designated as leaders autonomously navigate and patrol a configurable set of target locations. In contrast, drones designated as followers are assigned to track a specific other drone, maintaining a configurable separation distance throughout operation [4].

Independently of their assigned role, all drones continuously monitor their internal energy levels through a context-aware energy management mechanism. When the remaining charge drops below a set threshold, a high-priority *Charge* b-thread is activated by a complex constraint. It overrides the drone's current role and guides it to a designated charging station until its energy is fully replenished. This mechanism ensures that energy-critical behavior takes precedence over mission-specific tasks across all drone instances at all times [4].

The three used examples, the mobile phone, the water tank, and the drones scenario, are revisited in the evaluation chapter to demonstrate how the proposed toolchain can be applied in practice.

3 State Of The Art

This chapter surveys the scientific and technical landscape relevant to this thesis across three complementary domains: the modeling of feature trees, the comprehension of Behavioral Programming, and the implementation of graphical editor platforms. The following collection of works represents the most relevant contributions in these overlapping areas. The chapter first reviews the Universal Variability Language and the ecosystem of tools that enable formal architectural specification through feature modeling. It then examines Behavioral Programming and the specialized tools that have emerged to help developers understand the dynamic behavior of BP systems. Finally, examples utilizing the Graphical Language Server Platform are presented as modern, extensible frameworks for developing domain-specific diagram editors. To the best of our knowledge, no prior work has combined these three domains into an integrated approach, which motivates the novel contribution of this thesis.

3.1 Tooling Support for UVL-Based Feature Models

As the Universal Variability Language has established itself as a common exchange format for feature models across research and industry, a growing ecosystem of tools has emerged to support model authoring, visualization, and automated analysis [42]. The following subsections survey the most prominent solutions currently available, covering both dedicated language-server extensions for text-based editing and fully IDE that combine editing with graphical exploration and automated reasoning.

3.1.1 Universal Variability Language Server

The UVL Language Server (UVLS) is a LSP implementation specifically designed for UVL, providing editing assistance for the manual authoring of large feature models. Conforming to

the LSP protocol, UVLS integrates easily into any compatible editor. Integrations for Visual Studio Code and NeoVim are already available [43, 44].

UVLS performs multi-level static analysis as the user types, providing immediate feedback without requiring explicit validation steps. Syntactic validation leverages a tree-sitter-based parser [43] to detect grammar violations and pinpoint exact error locations. For semantic analysis, UVLS integrates the SMT solver Z3 [45] to identify modeling anomalies such as void models, dead features, contradictions, and redundant tautologies [43].

To further support model authoring, UVLS provides context-sensitive auto-completion that suggests valid group types, numerical attributes, and importable submodels based on the UVL grammar. Advanced syntax highlighting differentiates grouping keywords, feature identifiers, types, attributes, and constants, enhancing code readability. Reference handling enables users to locate all usages of a feature or attribute across cross-tree constraints and perform synchronized renames [43].

For interactive product configuration, UVLS provides a dedicated editor that supports decisions across all UVL domains and provides real-time validity feedback. Configuration choices can be rendered as inline code annotations within the source model, improving traceability [43]. Finally, UVLS can generate a Graphviz [46] `.dot` file from the feature model, which in environments such as VS Code is rendered and refreshed in real time as the user edits the source [43].

3.1.2 FeatureIDE

FeatureIDE is an open-source, Eclipse-based framework for Feature-Oriented Software Development (FOSD) that has become the de facto standard in both academic research and industrial practice. It supports multiple composition paradigms and implementation languages, including FeatureHouse, FeatureC++, Java, C++, Haskell, and XML, making it adaptable to diverse development contexts [47, 48].

The framework supports the complete FOSD workflow end-to-end. A graphical editor enables engineers to construct and maintain feature models, which are persisted in machine-readable formats suitable for automated analysis [47]. Figure 3.1 shows the mobile phone feature model introduced in Section 2.2.2. For configuration, a dedicated user interface allows users to select features for specific product variants, providing immediate validation of conflicting specifications. Background composition assembles program artifacts automatically as feature selections or implementations change. Compilation errors are propagated and annotated directly in the affected source files. FeatureIDE's extensible architecture enables researchers to integrate custom format parsers, type-checkers, and refactoring tools [47]. The effort required for advanced components often varies because they are not always generalizable across implementation strategies [49].

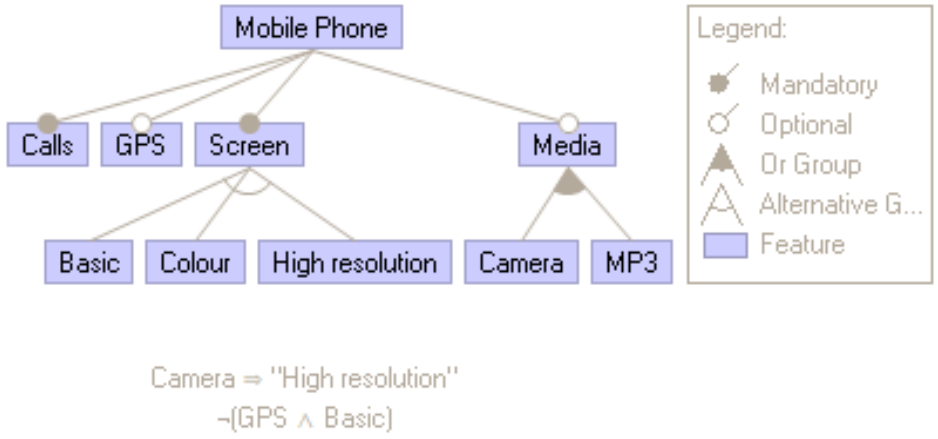


Figure 3.1: Graphical representation of mobile phone feature model rendered by FeatureIDE [48].

FeatureIDE supports UVL natively as an alternative textual format through a Clojure-based parser library and a thin integration layer that translates UVL elements into FeatureIDE's internal model representation [6]. Parsing errors are reported with precise line and column information, including semantic validation such as constraint reference checking handled by FeatureIDE itself. UVL's composition mechanism is fully leveraged: features from referenced submodels are visually distinguished by a sky-blue border and alias prefix. They remain read-only in composed views while being editable in their source submodels. However, UVL cardinalities (e.g., `[0..3]`) are not yet supported, requiring tool-specific metadata to be stored as generic feature attributes as a workaround [6].

3.1.3 flamapy.ide

`flamapy.ide` [50, 51] is a browser-based IDE built on the open-source `flamapy` framework [52, 53]. Rather than requiring users to install native applications or Python environments, the tool uses a fully client-side architecture, compiling `flamapy` to WebAssembly via Pyodide [54]. All computations execute directly in the user's browser. There is no dependency-management overhead, and sensitive feature models remain client-side without being transmitted to remote servers [50].

The IDE embeds a Monaco-based code editor with full UVL support. As users type, the editor automatically parses the document, providing syntax highlighting and error detection so that modeling mistakes surface immediately rather than being deferred to an explicit validation step. The user interface of `flamapy.ide` is illustrated in Figure 3.2 [50].

Additionally, feature models can be rendered as interactive tree graphs using D3.js [55], enabling intuitive navigation through the feature hierarchy. This tight coupling between textual and graphical representations ensures that any valid edit to the UVL source is immediately

reflected in the diagram [50]. For comparison, the mobile phone feature tree is shown in Figure 3.3 using `flamapy.ide`.

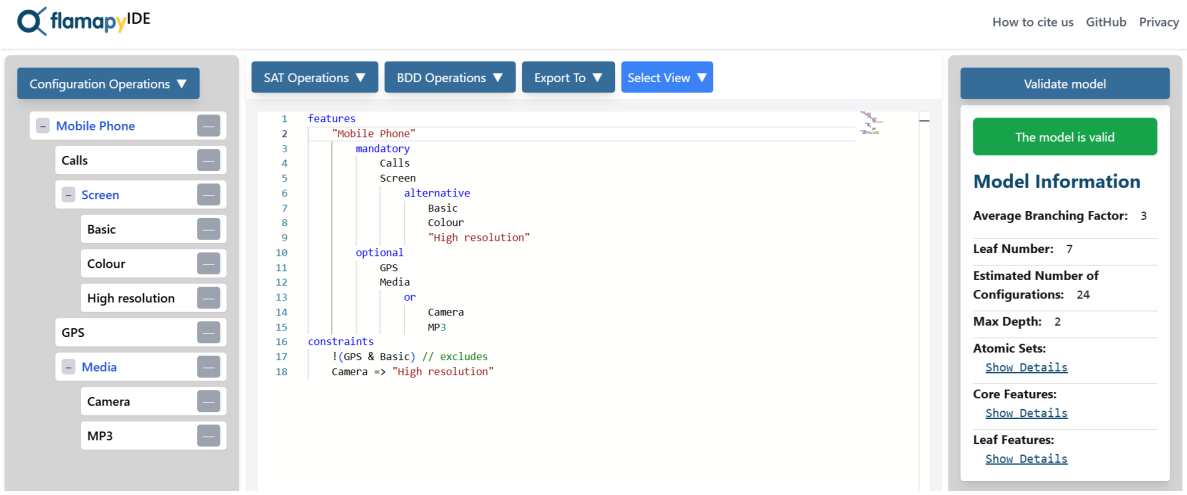


Figure 3.2: The `flamapy.ide` user interface with the UVL code editor [51].

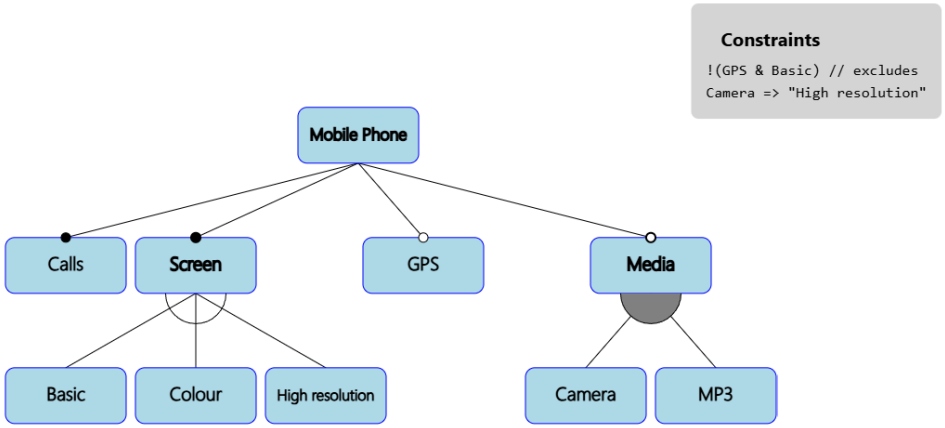


Figure 3.3: Graphical representation of mobile phone feature model rendered by `flamapy.ide` [51].

Beyond editing and visualization, `flamapy.ide` exposes automated reasoning through dedicated analysis backends. SAT-based routines use propositional satisfiability solvers to check model satisfiability and to count valid configurations. BDD-based operations implement Binary Decision Diagrams for efficient reasoning over feature models. These support configuration counting, core feature detection, and feature-inclusion probability estimation. An interactive product configurator guides feature selection step by step and validates each choice against model constraints, providing real-time feedback on configuration validity [50].

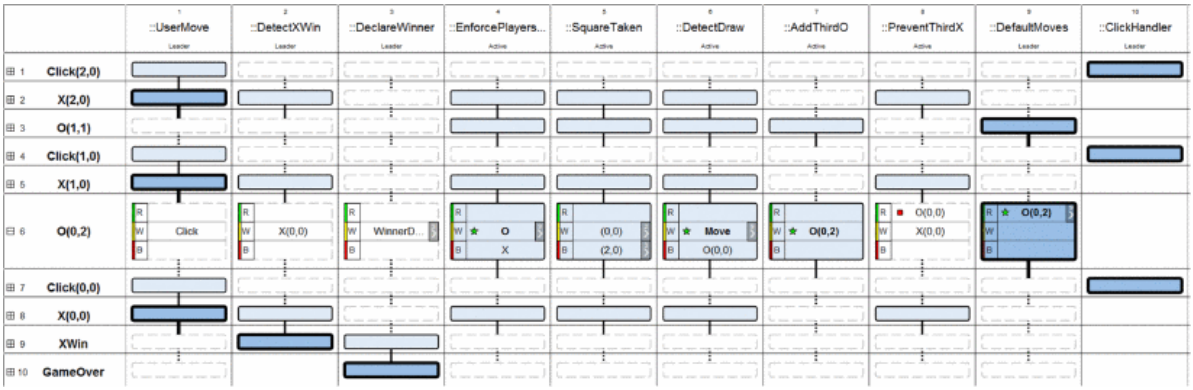


Figure 3.4: Showcase of TraceVis using a Tic-Tac-Toe game. Taken from [56].

3.2 Comprehension of Behavioral Programs

As Behavioral Programming applications grow in size, understanding why certain events are selected while others are blocked becomes increasingly difficult. The parallel execution of multiple b-threads, combined with complex event interaction rules, makes it difficult for developers to reason about program behavior or diagnose failures. To address this comprehensibility challenge, three distinct tools have emerged. Each targets a different phase of the development lifecycle and offers complementary capabilities for understanding BP systems.

3.2.1 Trace Visualization

TraceVis was proposed by Eitan et al. [56] and featured alongside the original Behavioral Programming paper [1]. It is a web-based, interactive grid display that shows the state of a behavioral program over time (Figure 3.4). Synchronization points are organized along the rows, while b-threads are organized along the columns, ordered by priority. Each cell represents the state of a b-thread at a given synchronization point, indicating which b-events are requested, waited-for, or blocked. Visual cues, such as green stars for selected events and red squares for blocked ones, help developers track the execution flow. To handle larger applications, TraceVis provides grouping by b-thread class and filtering mechanisms that hide inactive threads. The tool is designed for the Java implementation of BP, BPJ [57], which generates a trace file at the end of a run. Upon importing this file, TraceVis reconstructs the execution history and visualizes it in the browser [56]. As a result, TraceVis is strictly a post-execution analysis tool and cannot control or interrupt a live program.

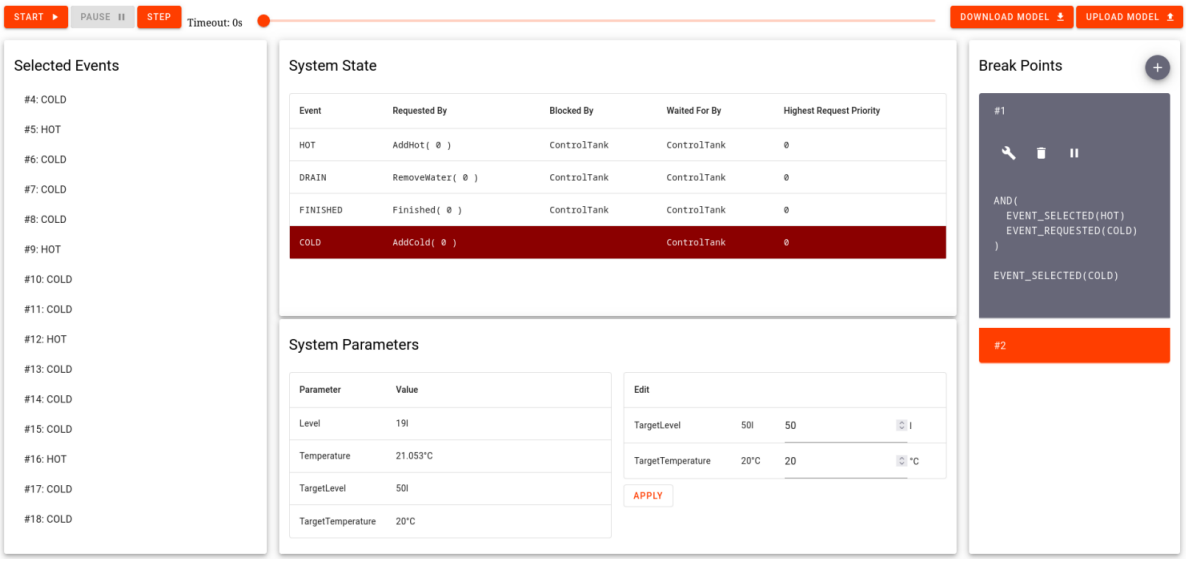


Figure 3.5: User Interface of the BP Runtime Debugger. Taken from [2].

3.2.2 Runtime Debugger

To address the limitations of TraceVis, a runtime debugger for BP was subsequently proposed [2]. It is a web application with a table-like structure for visualizing the system state, which is illustrated in Figure 3.5. Rather than consuming a post-run trace file, it hooks directly into the executing program and streams state data to the browser at every synchronization point. The data includes b-threads, b-events, and additional parameters. The approach provides runtime controls similar to those in symbolic debuggers, including the ability to pause execution and set conditional breakpoints. These breakpoints can be chained to halt execution only if a specific sequence of conditions occurs across consecutive synchronization points. Additionally, the debugger adopts a *models@run.time* approach to incrementally construct an abstracted runtime model in memory. This model enables time-travel debugging, allowing developers to navigate to arbitrary past system states and trace a bug's origin. Recorded traces can be exported and re-imported for side-by-side comparison with a live run [2].

3.2.3 Feature Modeling for Behavioral Programming

The framework FMBP [4] addresses comprehensibility at the architectural design phase before writing any behavioral code. Rather than analyzing a running or completed program, it provides a formal model of the expected system structure. FMBP is a Python framework that extends UVL to represent BP as an Architecture Description Language, modeling b-threads as structural features and their events as feature attributes. This formal representation enables automated SMT-checking and type verification before implementation, surfacing logical contradictions and constraint violations early. At runtime, FMBP connects the feature model to the executing program through two components. The `ConsistencyChecker` compares

the live system state against the expected model structure, while the `BPConfigurator` reconfigures the behavioral layout in response to dynamic environmental context [4]. Together, these mechanisms provide an overview of expected system interactions across the entire development lifecycle.

As part of the framework, FMBP extends the UVLS to support BP [58]. B-threads are represented as features, while b-events are modeled as composite feature attributes nested within their respective b-thread features. Each event attribute carries explicit properties for `requested`, `blocked`, `waited_for`, and a `priority` value [4].

Explicit type annotations are required throughout the model to prevent ambiguity and ensure consistency. A b-thread feature must contain an attribute `type 'BThread'` to indicate its role, while event attributes must be tagged with `type 'BEvent'`. The extended UVLS enforces strict name-type consistency, ensuring that any parameter passed into a constraint function carries a `BEvent` type annotation wherever it appears. Semantic errors, such as an event nested inside another event, are surfaced as immediate IDE errors [4].

The extension also defines aggregate constraint functions that translate into SMT-solver expressions for automated verification. Three basic functions are `requested()`, `blocked()`, and `waited_for()`. They query the model to determine whether any b-thread exhibits a specific behavior for a given event. Two specialized constraints support reasoning about event interactions. `conflicting()` enforces mutual exclusion among an arbitrary number of events. `selected()` verifies that a specified event is guaranteed to be triggered by analyzing the collective effect of all b-threads' priorities, requests, and blocks [4].

To support context-awareness, FMBP introduces two special top-level features for modeling dynamic behavior and system configuration. The `Env` feature holds variables representing runtime environmental state such as sensor readings, system parameters, or external events. The `Config` feature provides a top-level container for static configuration parameters that control the behavioral layout independent of runtime context. Both features can be referenced in constraints to enable context-sensitive behavior selection [4].

3.3 Existing GLSP-Based Diagram Editors

To understand the practical scope of GLSP and the architectural conventions that have emerged around it, this section examines concrete tools and applications built on top of the framework. First, the reference implementations maintained by the GLSP project are described, establishing baseline patterns and best practices. Next, BigUML is analyzed as a more ambitious third-party tool. The section closes with a discussion of a prototype that extends GLSP's protocol and architecture to support real-time collaborative modeling, thereby illustrating the framework's extensibility for novel use cases.

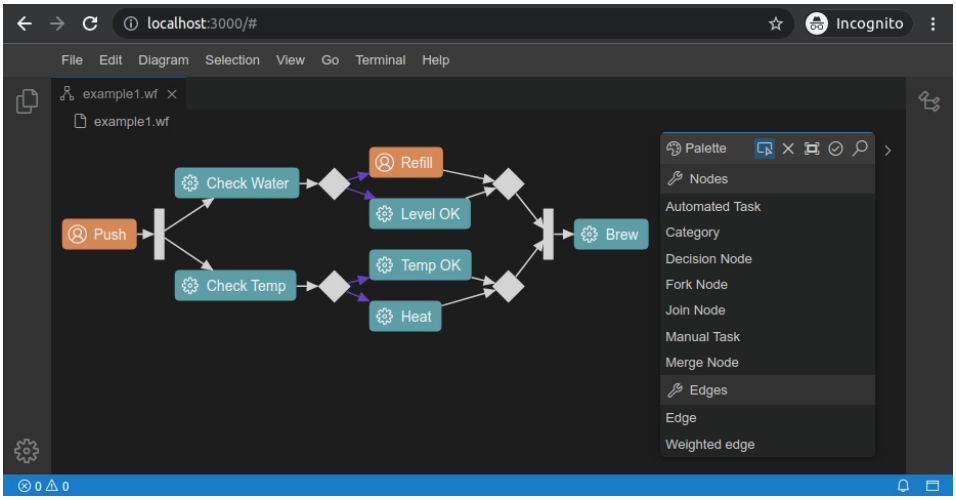


Figure 3.6: GLSP Workflow diagram. Taken from [61].

3.3.1 Example Projects of the GLSP maintainers

Server and client frameworks. To lower barriers to entry for GLSP users, the project provides two production-ready server implementations alongside its client library. The Java server framework integrates tightly with EMF, enabling developers to leverage existing metamodels and model-manipulation logic without reimplementing them [30]. The TypeScript server framework, in contrast, executes natively on Node.js and enables a homogeneous full-stack development experience when combined with the TypeScript client [31].

On the client side, GLSP builds upon Eclipse Sprotty, a mature graphics library that transforms abstract graph models into interactive SVG visualizations rendered in the browser [59]. Crucially, GLSP extends Sprotty's base graph schema (`SGraph`) into its own `GModel` while deliberately preserving direct access to the underlying SVG and CSS rendering layer. This design choice grants tool developers complete control over the visual representation of shapes, connectors, and interaction behaviors, enabling high degrees of customization without the need to fork the framework [5].

For deployment and integration, GLSP employs a clear architectural separation between platform-independent editor logic and host-application-specific concerns through dedicated integration modules. Platform-specific adapter packages enable the same diagram editor to be deployed into Eclipse RCP [60], VS Code [33], Eclipse Theia [34], or any plain web application [32]. This approach allows developers to reuse the same core editor logic and diagram definitions across multiple targets [5]. Moreover, the Node.js server implementation enables developers to run GLSP fully in the browser [5].

The Workflow Diagram example. The primary reference implementation for GLSP is the Workflow Diagram example [36]. It is a simplified flowchart editor designed to serve as the

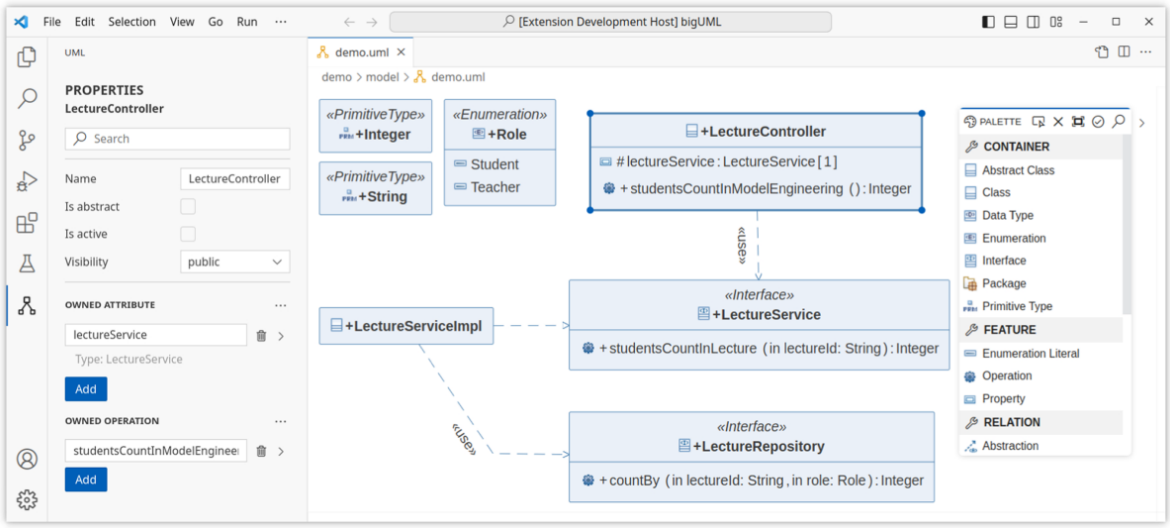


Figure 3.7: BigUML class diagram. Taken from [35].

canonical demonstration of GLSP's capability set. The example implementation includes the usage of all previously described server frameworks and client integrations. The workflow diagram showcases a comprehensive set of node and edge types, interaction patterns, and editor features that collectively exercise the full range of GLSP's architectural capabilities. In Figure 3.6, an example of the workflow diagram editor is shown. The example is fully open-source and serves as a blueprint for custom implementations, allowing developers to understand both the GLSP architecture and best practices through a single, authoritative example.

3.3.2 BigUML

BigUML is a proof-of-concept UML modeling tool built on GLSP that validates the framework's viability for constructing large-scale, specification-driven diagram editors [35]s. Implementing comprehensive web tooling for UML is notoriously demanding because the specification defines numerous structurally distinct diagram types, each with unique visual notations and semantic rules. BigUML demonstrates that GLSP can meet this challenge while delivering advanced end-user features, all organized into a modular architecture of core services, tool-level features, and diagram-specific modules [35].

The central architectural concept that allows BigUML to handle multiple UML diagram types over a shared source model is Source Model Representation Separation. The same underlying model element may need to be rendered differently depending on the currently active diagram type. When a diagram is saved, BigUML persists the diagram's representation type, for example, a class diagram and a communication diagram, to the file system alongside the source model. When the model is subsequently read, the model server supplies this

representation identifier to the GLSP server, which uses it to load the correct language-specific diagram module. User requests are then dispatched exclusively to that module, ensuring that distinct diagram representations share source data without interfering with each other's visual rules or behavioral constraints [35].

3.3.3 Real-Time Collaborative Modeling in GLSP

Real-time collaborative modeling in GLSP has been achieved by extending the protocol and architecture to support multiple users interacting concurrently on the same diagram [62, 63]. One such prototype uses Visual Studio *Live Share* as the data exchange layer and implements a single-server architecture. A designated host machine runs the authoritative GLSP server holding the current model state, while guest clients proxy their edits through the host. When a guest performs an edit, their action is forwarded via *Live Share* to the host client, which proxies it to the server. The resulting update is then broadcast to all participants. Since only one model state ever exists at any time, costly merge algorithms such as operational transformation are unnecessary. To maintain safe undo-redo semantics in this shared context, the server was extended with a `CommandStackManager` that maintains independent command stacks per user. This enables each participant, identified by their *Live Share* subclient ID, to undo only their own actions without affecting others' work [62].

Rather than enforcing restrictive element-level locking, the system resolves conflicts by allowing the most recent operation to succeed, and mitigates unintentional interference through rich visual awareness feedback. These awareness cues are transmitted via a dedicated `CollaborationAction` message type sent directly between clients, bypassing the server. They include live mouse pointers with participant name labels, dashed viewport outlines showing each participant's current view, and color-coded selection indicators on shared elements using each participant's assigned *Live Share* color. Usability studies with BigUML and the Workflow Diagram confirmed that the system scales well for typical collaborative group sizes, with session performance determined primarily by the rate of concurrent edits rather than by model size or complexity [62].

3.4 Research Gap

The preceding survey reveals three critical gaps that emerge precisely where Behavioral Programming, feature modeling, and graphical editor frameworks intersect. While each domain has matured significantly in isolation, the integration of formal architectural specification with visual modeling and runtime comprehensibility has received limited attention. The following open problems, derived directly from the state-of-the-art analysis, motivate our research contributions presented in this thesis.

Extensible GLSP architecture for UVL. BigUML demonstrates that GLSP can support multiple diagram types within a single tool, but achieves this by bundling all diagram types into one monolithic extension. As a result, individual diagram types cannot be developed or deployed independently of one another. How to design a modular GLSP-based architecture that permits a core diagram extension to be augmented by separately developed domain-specific extensions remains unexplored.

Graphical tooling for extended UVL. The graphical tools currently available for UVL, such as `flamapy.ide` and FeatureIDE, are built around the standard feature-modeling use case. Domain-specific extensions to UVL, such as those introduced by FMBP for Behavioral Programming, add new language constructs, types, and semantic constraints that go beyond what these general-purpose tools support. Whether such extensions can be integrated into existing tooling without causing incompatibilities with the broader UVL ecosystem remains an open question.

Runtime integration of FMBP. FMBP extends UVL to provide formal architectural comprehension of Behavioral Programming at the design phase, a valuable contribution addressed by none of the runtime comprehensibility tools surveyed in Section 3.2. However, FMBP in its current form operates entirely at the textual level and provides no visual representation of the architectural models. Moreover, the framework only partially employs the *models@run.time* paradigm because synchronization is strictly unidirectional. Changes to the textual UVL model are directly reflected in the running behavioral program, whereas propagating runtime state back into the static architectural model is not supported. Therefore, the gap between the static architectural model and the dynamic behavior of the running program remains open.

Runtime integration of GLSP for model synchronization. Existing GLSP-based collaborative work focuses on enabling multiple users to edit the same diagram in real time by proxying interactive client edits through a host. This real-time collaborative approach does not apply to the intended use case of programmatic runtime model synchronization, as it relies on interactive client edits rather than on automated, server-side synchronization with a running system. Consequently, it remains to be investigated how GLSP's protocol and server architecture can support runtime integration patterns that enable continuous synchronization between UVL models and the live execution state.

Thesis Contributions. We propose a modular, extensible GLSP-based graphical editor for the extended UVL together with a runtime integration mechanism to explore synchronization between UVL models and executing Behavioral Programs. In the following chapters, we will address the process of designing, implementing, and evaluating this tool.

4 Concept

In this chapter, we present the conceptual design of a graphical editor for managing UVL-based feature models and its extension to contextual behavioral programs. It outlines the core architectural decisions, the integration points for behavioral programming concepts, and the runtime synchronization mechanisms that enable *models@run.time*.

4.1 GLSP Architecture & Framework Selection

This section explains the architectural choices and framework selections made for implementing the graphical UVL editor. It connects those choices to the thesis goal of providing a GLSP-based editor extended for context-oriented behavioral programs. It outlines the separation of responsibilities between server and client components, the role of the graphical model in client-server communication, and the rationale for selecting the server and client technologies.

4.1.1 Server Framework Selection: Java

GLSP provides two production-ready server framework implementations [5], one in Java [30] and one in TypeScript [31]. The Java server framework supports multiple source model strategies. The most common strategy is to use the GLSP-native graphical model (GModel) as the primary data structure for both semantic and visual information. The GLSP maintainers demonstrate this approach in the workflow example [36]. An alternative is to integrate an EMF-based semantic model directly into the server. The Java framework provides several variants of EMF-based integration: direct EMF model integration, EMF integration paired with a separate notation model, and integration with an EMF.cloud model server for centralized persistence and external semantic authority [5]. The Java server further benefits from a mature ecosystem of multiple examples, reducing implementation friction and providing

proven patterns for complex model operations. The TypeScript server framework, while offering a homogeneous full-stack development experience, lacks the same depth of support for source models. It supports only a single source model strategy in which the GModel serves as the primary data structure for both semantic and visual information [31].

We choose the Java server framework for this work, although it does not adopt the EMF-based source model. The Universal Variability Language does not provide an EMF domain model (an *Ecore* model). The framework's EMF-based source model strategies require such an *Ecore* model. However, the UVL maintainers have developed the `java-fm-metamodel` package [64]. It is a Java library that parses UVL files using the `uvl-parser` and materializes them into structured Java objects suitable for programmatic manipulation [65]. This library enables the GLSP server to load UVL models directly, modify them during editing operations, and re-serialize them back to UVL files, providing a stable foundation for model manipulation. This is an advantage over the TypeScript server, which would require re-implementing the parsing and model-manipulation logic from scratch. This server-side foundation supports RQ1 by making graphical visualization of UVL feature models technically feasible within the editor architecture.

This work adopts a custom dual-model approach in which the server maintains two parallel representations. The semantic layer consists of `java-fm-metamodel` objects that represent the UVL source model and serve as the authoritative source for model modification. The visual layer consists of a GModel derived from this semantic representation. The server is responsible for transforming any source model into a GModel that contains the visual information transmitted to the client [5]. To persist notational information across editing sessions, the server serializes the GModel to JSON, saving node positions, dimensions, and visual groupings in a separate file alongside the UVL source file. When a model is loaded, the server reads both the UVL source file and the corresponding GModel file, reconstructing the previous visual representation. Changes made by the user in the client are translated into semantic modifications on the `java-fm-metamodel` objects, which are then reflected in the GModel and persisted back to both the UVL source file and the GModel JSON file.

Layout metadata, such as element positions and sizes, could potentially be encoded as feature attributes. The downside of embedding notation into the semantic `.uvl` file is that it would conflate the two concerns. This would invalidate the file for any non-GLSP tooling and include attributes that do not belong in other use cases.

4.1.2 Client Framework Selection: VS Code

GLSP provides platform integration packages for multiple integrated development environments as described in Section 3.3.1. We select VS Code for client deployment for several reasons. It is the most widely adopted IDE in contemporary software development practice, reducing adoption barriers for end users [66]. The platform also supports creating and

distributing extensions via the `.vsix` format, enabling a clean separation between the base UVL tool and its BP-specific extension [67]. Finally, the GLSP integration for VS Code is stable, well-documented, and actively maintained, providing a reliable foundation for implementation [33]. As this work focuses on establishing a proof of concept, a single environment is sufficient. Broader IDE support may be explored in future work.

4.1.3 Feature Division

A central architectural decision is to separate base UVL functionality from BP-specific features into two distinct, independently distributed packages. The base UVL tool delivers a standalone `.vsix` extension providing general-purpose feature modeling capabilities, while the UVL-BP extension builds upon it by adding behavioral programming semantics. The separation ensures that users who work exclusively with UVL have no dependency on BP-specific tooling, and that the UVL-BP extension can evolve independently without affecting the base editor. The package structure that realizes this separation is described in the following Section 4.1.4.

Base UVL tool. The base tool visualizes the feature hierarchy by presenting feature names, parent-child relations, and constraints in a structured tree view. It renders group relations such as *alternative* and *or*, and it supports common cross-tree constraints such as *requires* and *excludes*. Features may carry arbitrary key-value attributes, and the editor also supports feature-level and group-level cardinalities. Its core editing capabilities cover creating, updating, and deleting features, relations, constraints, and attributes. The graphical representation adheres to the design requirements outlined in Section 4.4.

UVL-BP extension. The UVL-BP extension depends on the base UVL tool and introduces additional model elements and runtime capabilities specific to behavioral programming. It adds two root features, `Env` and `Config`, that represent the model's context. A new b-thread feature type is introduced to represent behavioral threads as first-class model elements. Different kinds of b-events, namely *requested*, *waited-for*, and *blocked*, may be attached to a b-thread. BP-specific constraints, as defined by Felber and Götz [3], are supported via the existing constraint representation of the base tool, thereby extending its semantics without requiring additional model structure. Finally, the extension realizes a *models@run.time* approach by linking the graphical model to a live behavioral program, reflecting its execution state directly within the editor. We discuss this integration in detail in the following Sections 4.2 and 4.3. This feature split supports RQ2 by separating the base visualization from BP extensions while maintaining an extensible architecture.

4.1.4 Intended Package Structure

We intend to realize the described separation through a layered package structure that keeps the UVL core reusable while isolating BP-specific extensions. On the server side, three main packages are intended. The `uvl.metamodel` package contains the adapted parser and metamodel for loading and materializing UVL source files. The `uvl.glsp` package provides the base GLSP infrastructure for UVL and concentrates all reusable tools, bindings, and shared editor logic. The `uvl.bp.glsp` package extends this infrastructure for behavioral programs and adds only the BP-specific model elements, constraints, and runtime interactions. This structure is designed to minimize code duplication by letting the package reuse as much of the common server implementation as possible.

On the client side, we maintain a similar separation. There are packages for the base UVL and for the BP extension, each containing the client-side logic relevant to its context. The VS Code integration code is shared and not duplicated between both variants. Instead, profile-specific packaging assembles generated VS Code extensions from the shared client modules and includes the package set that matches the selected profile. This allows the repository to produce multiple `.vsix` artifacts from one codebase while keeping the common integration and rendering code in a single place. This package structure advances RQ2 by making it possible to add new variants and analyses without duplicating the shared editor foundation.

4.2 Integrating Behavioral Programming into UVL

This section describes, at a conceptual level, how Behavioral Programming can be integrated into the UVL-based language and its available tooling. The objective is to extend the language and tools so that models can express BP event constraints and BP-specific top-level features while preserving UVL's core philosophy and backward compatibility.

4.2.1 ANTLR grammar extensions: `uvl-parser`

As described in Section 3.2.3, FMBP already implements an extension of UVLS to represent BP-specific semantics. UVLS is based on the tree-sitter parsing library [43], which did not require any grammar modifications to support the BP constructs [4, 58]. For this work, the `uvl-parser` is used instead [65, 68], which is based on ANother Tool for Language Recognition (ANTLR) [69]. The `uvl-parser` library is also used in the UVL integration of FeatureIDE [48]. ANTLR is a parser generator that takes a formal grammar as input and produces a parser in a target host language [70]. The `uvl-parser` provides the ANTLR grammar files that define UVL's concrete syntax and supports Java as a target language besides Python and JavaScript [68]. Although both the tree-sitter-based and ANTLR-based parsers represent the

same language, their definitions of the concrete syntax differ. The BP constructs introduced by Felber and Götz [3] are not currently supported by the `uvl-parser`, so the grammar must be extended to accommodate them.

According to the ANTLR grammar file, UVL permits a single top-level feature at the model root [68]. An example of such a grammar rule is shown in Listing 4.1. BP models extend this by requiring two additional, named top-level features at the root: `Env` and `Config`. Due to the grammar definition, it currently rejects models that place `Env` or `Config` next to the common feature model root, for example, the UVL model shown in Listing 2.2. To support BP while preserving compatibility, the grammar must be extended so these two required top-level features are accepted in a backward-compatible way. Importantly, this allowance must be limited to the default single top-level feature plus the two BP-specific sections, and must not permit arbitrary additional top-level features. Standard UVL models that do not include these features should continue to parse unchanged when BP parsing is not enabled.

```
1 featureModel:
2    namespace? NEWLINE? includes? NEWLINE? imports? NEWLINE? features? NEWLINE?
         constraints? EOF;
3 ...
4 features: FEATURES_KEY NEWLINE INDENT feature DEDENT;
```

Listing 4.1: Excerpt from the `uvl-parser` grammar showing the restriction for a single top-level feature.

The existing constraints defined in the `uvl-parser` grammar are shaped around classical UVL constraint forms. They include Boolean and mathematical combinations, common expressions and functions (e.g., sum aggregates), and cross-tree relations between features [68]. The base grammar already includes single-reference constraint shapes for these use cases so that this approach can be reused for BP-specific single-reference constraints such as `requested`, `waited_for`, `blocked`, and `selected`. Reusing that approach may require lexical disambiguation for the event-role names so they can appear unambiguously as attribute identifiers. In contrast, `conflicting` is a true multi-reference constraint that binds several feature references simultaneously and is not covered by existing single-reference forms. To support it, an explicit multi-reference production must be added to the grammar. These BP-specific constraints should be grouped as a distinct alternative, accepted only when BP parsing is enabled, so they do not interfere with the existing constraint forms. B-threads and b-events themselves are already representable in the grammar as attributes of features and do not require changes to the existing rules. Their semantics can be handled by the GLSP server or other tooling after successful parsing. Taken together, these grammar changes provide the syntactic basis needed to extend the language in a way that advances RQ2.

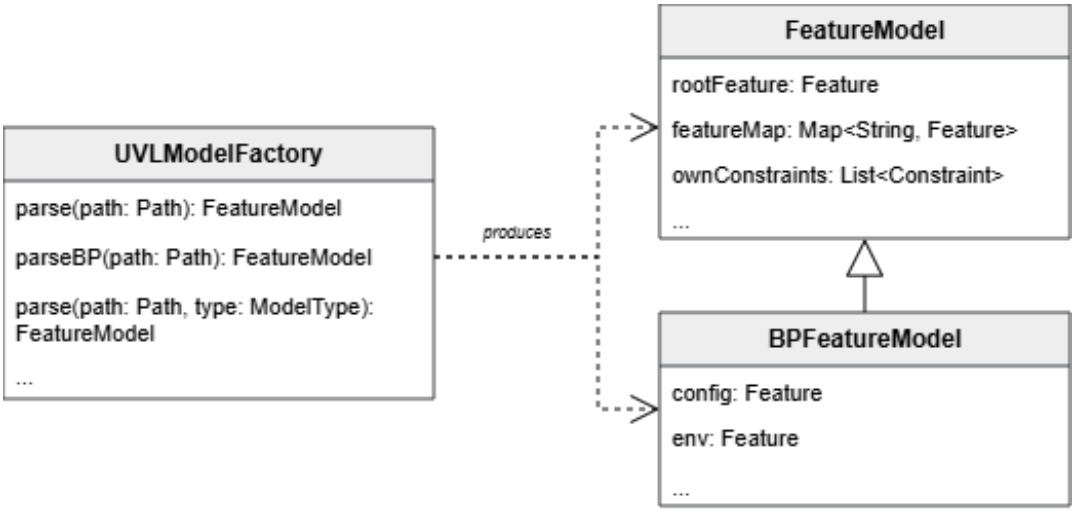


Figure 4.1: Simplified class diagram showing `FeatureModel` as the base class and `BPFeatureModel` as its extension; both classes are produced by `UVLModelFactory`.

4.2.2 Java metamodel extensions: `java-fm-metamodel`

The `java-fm-featuremodel` hosts the Java source model classes and the parser-to-model factories that translate ANTLR parse trees into typed feature model objects [64]. Due to its coupling with the `uvl-parser`, it must be extended to support the new grammar constructs introduced for BP.

The metamodel must distinguish between an existing class, `FeatureModel`, and a BP-specific class, `BPFeatureModel`. It allows the GLSP server to be deployed as two distinct variants, one targeting plain UVL authoring and one targeting BP-enhanced authoring, each activating only the capabilities relevant to its context. Therefore, we create the `BPFeatureModel`, which subclasses the base `FeatureModel`. The class must include additional fields for the `Env` and `Config` top-level features, enabling direct access to these contextual elements. Consumers that work exclusively with the base class remain entirely unaware of the BP extension and require no modification. Figure 4.1 displays a concept of the extended setup.

BP introduces constraint semantics that cannot be represented by the existing constraint classes [64]. An abstract base class can be added to the metamodel to represent the general concept of a BP-specific constraint, with concrete subclasses for each specific constraint type. Additionally, the `Conflicting` subclass should support multiple references.

The model factory and parser listener must be extended to instantiate these objects when BP parse nodes are encountered. For example, the listener must recognize the new top-level feature rules and attach them to the `BPFeatureModel` when BP parsing is enabled. The existing unit tests for the `java-fm-metamodel` should be extended to cover the new BP constructs.

The extended metamodel supports both parsing of default UVL models and parsing of new BP-enhanced models, thereby answering RQ2 of this thesis.

4.3 *Models@run.time* via FMBP

A central goal of the proposed tool is to realize the *models@run.time* paradigm [7, 26]. In the context of behavioral programs combined with UVL feature models, the graphical representation presented to the user through the GLSP editor must not only serve as a static configuration artifact. Rather, it must continuously mirror the live state of the executing behavioral program. Conversely, modifications the user applies to the graphical model should propagate to the running system.

FMBP provides the foundational layer for this integration [4]. It is the only existing framework that combines the execution of behavioral programs with a UVL-based configuration mechanism, making it the ideal candidate for realizing the *models@run.time* approach in this graphical context. The following section describes the conceptual design for extending FMBP to provide the runtime visibility and synchronization features required for an interactive model representation in the GLSP editor.

4.3.1 The Runtime State Visibility Problem

During execution, FMBP maintains all dynamic state internally. In particular, two categories of runtime information are relevant for a live model representation.

Dynamic environment attributes. As described in the previous sections, the UVL feature model includes an `Env` feature whose variables capture measurements of the system's operational context. While we initialize these values from the model at startup, the running FMBP framework continuously updates them in a local Python object [40]. FMBP never modifies the underlying UVL files during execution. The current values exist exclusively in memory and are invisible to any external observer.

Selected B-Events. The BP paradigm centers on synchronizing b-threads through the declaration and selection of events. Whenever the FMBP runtime selects a b-event for execution, this constitutes a meaningful behavioral transition. Currently, in the examples provided by FMBP, such selections are only reported to the console as log output [40]. The GLSP server has no means to observe these transitions and therefore cannot reflect them in the graphical model.

Both categories represent live runtime information that the GLSP server requires in order to uphold *models@run.time*. Since GLSP operates as a separate process and has no direct access to the FMBP Python runtime's object space, we must establish a communication mechanism.

That mechanism must proactively publish the runtime information. By maintaining the GLSP server as an independent modeling component, the tool preserves its role as a general-purpose graphical editor. This makes it extendable to multiple runtime systems without tight coupling to FMBP's internals. This separation of runtime state from the graphical editor defines the synchronization problem that underpins RQ3.1 and RQ3.2.

4.3.2 Server-Sent Events as a publication mechanism

A suitable solution for bridging the gap between the FMBP runtime and the GLSP server is Server-Sent Events (SSE). SSE is a lightweight, unidirectional HTTP-based streaming protocol in which a server pushes a stream of text-encoded events to any connected client [71]. It is well supported, requires no custom protocol negotiation, and is straightforward to implement in Python [72]. Crucially, it aligns with the problem's directional nature. The FMBP runtime is the sole authoritative source of truth for live state, and the GLSP server is a passive consumer that must apply updates as they arrive.

Under this approach, the FMBP runtime exposes an SSE endpoint. The GLSP server subscribes to this endpoint at startup or when a user initiates a connection, receiving a continuous stream of events. Upon receiving an event, the GLSP server applies the encoded state change to the internal model representation and triggers an update of the diagram. This keeps the visual model synchronized with the executing program without requiring polling or file-based communication. This publication mechanism provides the real-time update channel required to make RQ3.1 feasible in the graphical editor.

For the implementation, we use a single shared SSE port to enable concurrently running runtime models. This constraint is necessary because one GLSP instance cannot reliably maintain separate connection settings for multiple model-specific SSE ports. Therefore, each event carries the originating source file as metadata. The source file then acts as the routing key that determines whether a received event belongs to the current GLSP instance.

4.3.3 Extending FMBP: Two Complementary Mechanisms

To realize the SSE publication, we introduce two architectural extensions to FMBP. They are designed to be minimally invasive, preserving full backward compatibility with existing FMBP examples.

Decorator for Context Sources. We address the `Env` attribute problem by applying the *decorator* design pattern [73] to the existing context source abstraction in FMBP. The framework uses a `ContextSource` that is injected into the `ContextConfigurationProvider` to supply environment data to the runtime [4]. By wrapping the original context source in a

decorated class, every invocation of the used `get_data()` is intercepted [40]. The decorator first delegates to the source to obtain the current context snapshot, and then publishes a corresponding SSE to the GLSP server before returning the data to the runtime. The behavioral program itself requires no modification. The decorator is inserted at the provider's construction site, making it trivially opt-in.

To avoid redundant network traffic, the decorator performs payload deduplication. A new event is only emitted when the current snapshot differs from the previously published one. This prevents the GLSP server from processing unnecessary updates when environment values are stable.

BPConfigurator Extension Mechanism. We address the b-event selection problem by a formal extension mechanism in the `BPConfigurator`, the central orchestration class and core component of FMBP [4]. `BPConfigurator` subclasses `BProgramRunnerListener` from the `BPpy` package [74] and observes the program via two hooks. These are `starting(...)`, which is invoked once at program start, and `event_selected(...)`, which is invoked for every selected b-event. To integrate extensions, `BPConfigurator` is extended by passing a list of `BPConfiguratorExtension` objects at construction. The extension protocol exposes the same hooks so extensions align with the configurator's lifecycle. A concrete implementation of `BPConfiguratorExtension` should publish an SSE on each `event_selected` call. As with the decorator, extensions are registered at construction time and require no changes to core runtime logic. Together, the decorator and configurator extensions provide the runtime hooks needed to observe both contextual updates and event selection, thereby directly advancing RQ3.1.

4.3.4 Interaction Flow

The combined interaction between FMBP, the SSE layer, and GLSP is illustrated in the sequence diagram in Figure 4.2. Upon program startup, `BPConfigurator` invokes the `starting` hook of all registered extensions, allowing the SSE server to initialize. The GLSP server connects to the SSE endpoint and begins listening. During execution, each selected a b-event triggers an *event_selected* server-sent event, which the GLSP server uses to highlight the corresponding element in the model. Whenever the runtime samples a new context snapshot via the decorated context source, FMBP pushes a *context_update* event to the GLSP server, which updates the `Env` feature shown in the graphical model. Both scenarios directly advance RQ3.1.

User edits in the graphical model are written back to the underlying UVL files and are processed by the UVLS together with the `ConsistencyChecker` [4]. The current FMBP implementation restricts certain updates. Edits to the `Env` feature's measurement values are not applied to the live runtime [40]. Although the changes do not affect the live system, a user may persist them

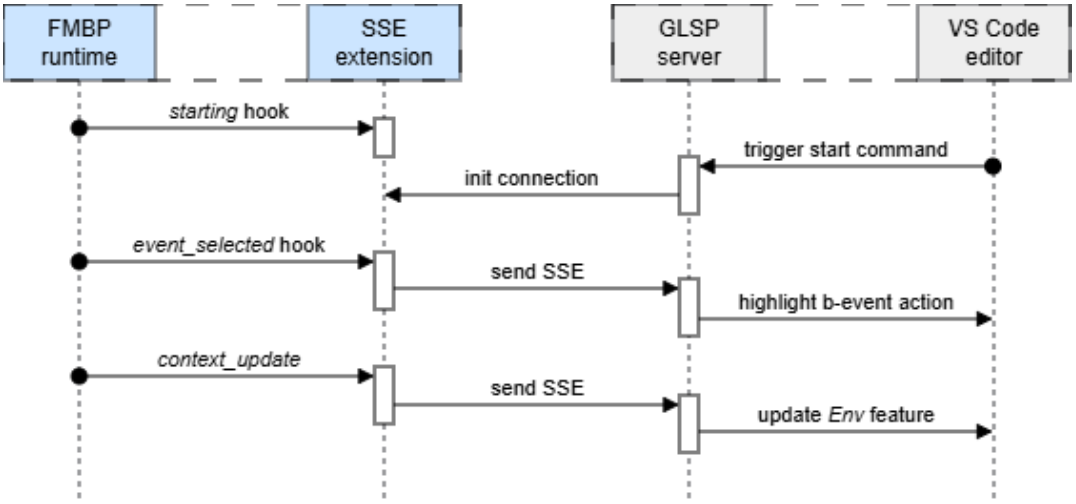


Figure 4.2: Sequence diagram showing the interaction between FMBP and GLSP.

to the UVL file using GLSP's save action. To improve performance, we do not automatically persist context updates. Variables explicitly designed for runtime reconfiguration are the attribute values of the `Config` feature. Edits to these values are applied to the live runtime after persisting them in the UVL file. This write-back path defines the synchronization mechanism that answers RQ3.2. This bidirectional coupling between the graphical model and the runtime state realizes the intended *models@run.time* approach described in Section 2.3.

4.4 Feature Model Visualization

This section defines the visual conventions for representing UVL-based feature models in the proposed editor. Each decision is motivated using the Physics of Notations [38] and, where instructive, contrasted with the conventions established by FeatureIDE [47]. The goal is to preserve the notation familiarity that domain experts bring from FeatureIDE while making the semantics specific to UVL-BP, namely behavioral elements, root contexts, and rich attributes, directly visible in the diagram.

4.4.1 Canonical node structure

Feature model editors have converged on a labeled-node tree as their canonical representation, and FeatureIDE exemplifies this convention. A labeled node maps unambiguously to a single feature, and parent-child edges represent the decomposition. Small decorators at the edge ends encode *mandatory* and *optional* relations. A set of edges that are part of an *alternative* or *or*-relation is grouped with a distinct circle sector. This one-symbol-per-construct mapping satisfies *Semiotic Clarity* and produces a layout that is immediately legible to practitioners trained on any standard notation. Departing from this topology without a strong

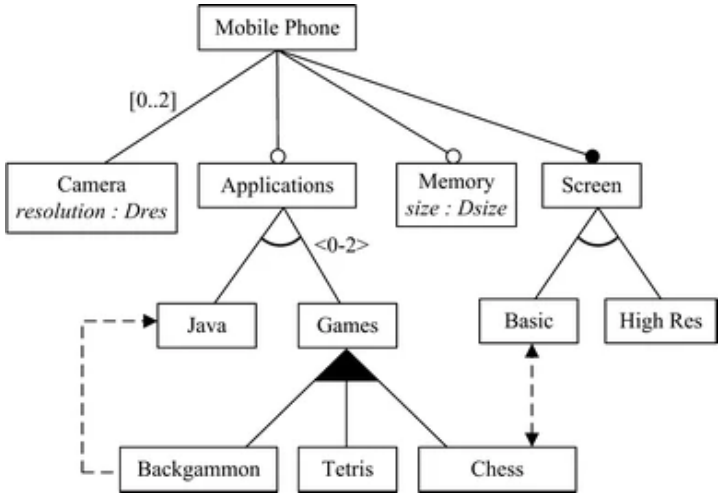


Figure 4.3: Example of feature model with attributes. Taken from [24].

semantic justification would violate *Cognitive Fit*. Such deviation would also alienate the target audience.

We therefore retain the node-and-edge structure shown in Figure 2.2. *Graphic Economy* is maintained by using a single node shape for all ordinary features. Additional visual channels, such as color, icons, and compartments, are reserved for element types whose semantics genuinely differ from a plain feature. Overloading the base symbol with decorative variation would reduce *Perceptual Discriminability*. It would also unnecessarily inflate the sign inventory. This canonical structure supports RQ1 by preserving a familiar and immediately legible basis for visualizing UVL feature models.

4.4.2 Attribute visibility: UML-style compartments

The representation of feature attributes in feature models lacks a unified standard across the literature and tools. Extended feature models vary widely in their approach. Some embed attributes directly within the feature node itself [24], as shown in Figure 4.3, while others place attributes in separate external nodes that are then linked to their representative features [21]. This diversity reflects the absence of consensus on the trade-off between notational economy and semantic transparency.

FeatureIDE externalizes attributes entirely. They are accessible only through a separate property view and invisible in the diagram itself. This can be seen in Figure 4.4. For a general-purpose feature model, this is a reasonable trade-off for diagram compactness. For UVL-BP, however, attributes are semantically load-bearing. They parameterize behavioral definitions and directly influence runtime behavior. Hiding them behind a panel severs the spatial connection between a feature and its behavioral configuration. This weakens *Semantic Transparency* and forces the user to reconcile two disconnected views mentally.

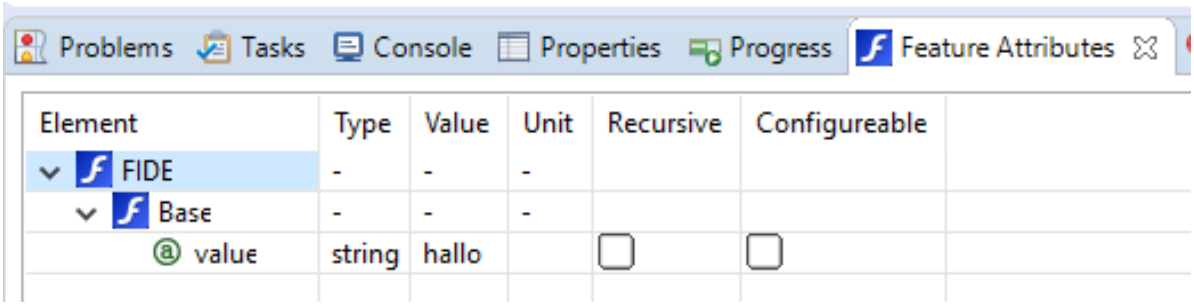


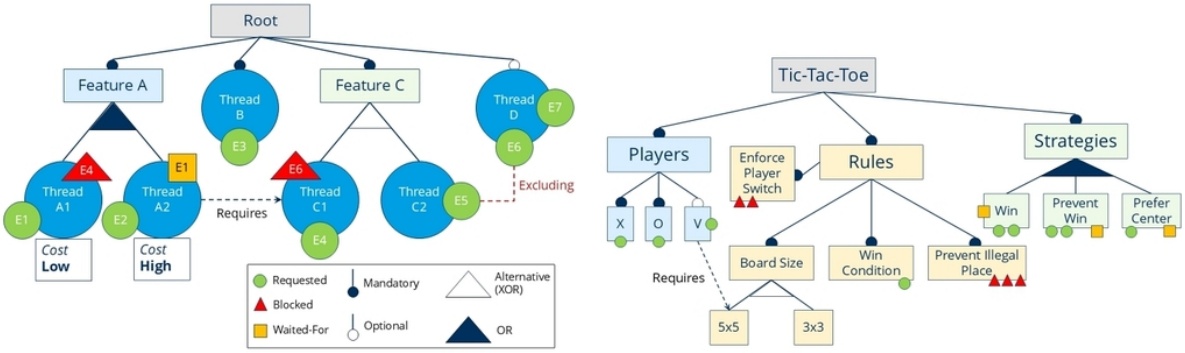
Figure 4.4: Feature Attributes View in FeatureIDE. Taken from [75].

We adopt a UML-style compartmental node form to address this. Each node consists of a header row and a lower compartment. The header row displays the feature name and an optional type glyph, which will be used in the following section. The lower compartment lists attributes one per line in a compact typeface. To preserve *Graphic Economy*, the attribute list uses reduced line spacing and a smaller font to conserve space. Attributes are shown directly in the diagram at all times, except where an attribute is explicitly represented in another form, as described in Section 4.4.2. Attribute lists are not dynamically collapsed or truncated. The diagram presents them plainly so users can read and act on them without opening secondary controls. All attribute modifications are made directly within the graphical view using inline editing tools. There is no separate property palette for attribute editing. The graphical representation therefore serves as the single authoritative editing surface, keeping model structure and attribute values co-located for clear *Cognitive Integration*. This compartment-based attribute view advances RQ1 by keeping semantically relevant data visible within the diagram itself.

4.4.3 Behavioral element types: b-thread and b-event

FeatureIDE's symbol vocabulary already distinguishes certain feature types through visual encoding. Abstract and concrete features, for instance, use different shades of color and border styles. This approach satisfies *Semiotic Clarity* by making semantic differences immediately visible without expanding the node-shape vocabulary. For BP, we must represent two new element types: b-threads and b-events.

Felber and Götz present two example visualizations of BP-augmented feature models [3], as shown in Figure 4.5. In the generic example (Figure 4.5a), b-threads are represented explicitly as blue circles attached to the tree. In the Tic-Tac-Toe example (Figure 4.5b), no separate visual distinction for b-threads is made. Hence, the two images show that a consistent semantic convention for representing b-threads in feature trees has not yet been established. In both examples, b-events are displayed as symbols attached to their parent feature. Green circles denote requested events, yellow squares represent waited-for events, and red triangles



(a) Generic feature tree (b) Tic-tac-toe Example

Figure 4.5: Different examples of feature trees with b-threads and b-events. Taken from [3].

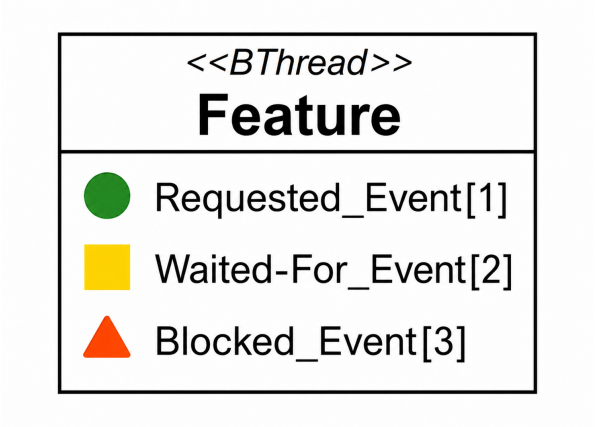


Figure 4.6: Conceptual depiction of a b-thread with all three b-event types included in the attribute compartment.

indicate blocked events. These event symbols are placed attached to the feature node to show the event-role relationship spatially.

We introduce b-threads to the editor as a feature with distinct runtime semantics. It is rendered as a standard compartmental node with an additional type label affixed to the header. This header decoration clearly marks the behavioral role without introducing a new node shape. The type reference thus provides *Perceptual Discriminability* while preserving the familiar node topology.

By contrast, we model a b-event as a specialized attribute within a feature rather than as a standalone feature node. Each b-event carries sub-attributes, such as name and priority, which would be difficult to display if events were represented only as attached symbols on the tree. Treating events as attributes allows the diagram to show event names and priorities inline in the attribute compartment while still using the distinctive icons (green circle, yellow square, red triangle) to indicate the event type. This keeps behavioral specifications spatially co-located with their owning feature and preserves *Semantic Transparency* without sacrificing

the expressive detail of event parameters. A sample b-thread with all possible b-event types is presented in Figure 4.6.

4.4.4 Constraint representation

FeatureIDE renders cross-tree constraints in a dedicated textual table below the diagram. This approach provides no spatial cue about which features are involved. Users must manually cross-reference the table with the tree. This weakens *Dual Coding* and increases cognitive overhead for constraint-heavy models.

We introduce a *binary constraint rule* as an explicit design boundary. Only binary relations ($n = 2$) are mapped to directed diagram edges. Constraints involving more than two features ($n > 2$) and complex mathematical expressions are offloaded to a supplementary textual list. This process of *symbolic offloading* keeps diagrammatic complexity bounded. Binary constraints get a direct visual representation that leverages spatial cognition, while the tool retains full expressive power in the textual form. The combination of both channels improves comprehension beyond what either representation achieves alone.

Concretely, we render binary *requires* and *excludes* relations in the diagram as dashed, directed cross-tree edges. Each relation involves exactly two identifiable nodes, and both types appear pervasively across use cases, making them ideal candidates for direct visual encoding. We collect all remaining constraints in a floating constraint list. This list is part of the graphical canvas and can be repositioned freely. Users who need close cross-reference with the tree can place the panel nearby. Others can move it aside when the focus is on the tree structure itself. This arrangement preserves *Graphic Economy* for the primary tree view while all constraints remain immediately accessible without leaving the editor. *Complexity Management* is achieved through spatial arrangement rather than through hiding. This avoids the information loss that a purely collapsible panel would introduce.

4.4.5 Root contexts: `Env` and `Config`

Standard UVL feature models have a single root feature from which all other features descend. The proposed tool extends this to the BP-specific environment by adding two additional top-level elements, `Env` and `Config`. These elements exist outside the main feature tree, and neither is a parent of nor a child of any feature. Instead, both hold attributes that represent global context variables. Constraints and behavioral definitions throughout the model may reference these variables, making it a priority to integrate them into the current structure. Forcing them into the tree layout would misrepresent their semantics and distort the hierarchy. For comparison, their role is analogous to that of the constraint list. Both represent globally relevant information that supplements the feature tree without being part of it in a structural sense.

The proposed design therefore treats `Env` and `Config` consistently with the constraint list. Both are rendered as floating, repositionable panels on the canvas. They use the same compartmental node form as ordinary features. The header row shows the element name and a type glyph, whereas the lower compartment lists attributes. Users can position these panels anywhere on the canvas, allowing them to be visually associated with the parts of the tree that reference their attributes or to be set apart when the focus is on the tree structure itself. This consistent treatment satisfies *Semiotic Clarity* because ancillary, globally scoped artifacts receive the same visual form. It supports *Cognitive Integration* by keeping the full model on a single canvas. No structural layout is imposed that the semantics do not warrant. At the same time, the distinct type glyphs in each header ensure *Perceptual Discriminability* between the two contexts. The encoding of BP-specific elements supports RQ1 and RQ2 by making behavioral concepts visible without abandoning the familiar feature model notation.

4.4.6 Runtime visualization and SSE handling

The editor must be able to visualize runtime updates emitted by the FMBP runtime as SSE, as described in Section 4.3. These payloads are consumed by the editor extension and translated into transient updates on the canvas. There are two complementary handling strategies for incoming SSEs.

First, the integration of context updates is a straightforward refresh of the affected node with the new attribute values. Second, we map selected b-events to a visual highlight on the owning b-thread feature node. When the SSE indicates that a b-event was selected, the server maps it to the owning feature and the b-thread feature that handles the event. The affected b-thread feature node receives a temporary highlight effect triggered by the client to draw the user's attention, for example by adding a glowing border to the node. These visual cues are transient and driven by the live event stream, so they communicate runtime activity without altering model contents. Both types conform to *Semiotic Clarity* by using distinct visual channels to provide runtime feedback. This runtime visualization advances RQ1 by making BP activity visible in the editor, and it supports RQ3.1 by allowing users to observe runtime updates in the graphical representation.

4.4.7 Multi-model views and VS Code integration

We leverage Visual Studio Code's multi-pane user interface to display multiple models simultaneously. Formally, VS Code exposes this capability through *Editor Groups* and the *Split Editor* command (`workbench.action.splitEditor`) [76]. Editor Groups allow users to arrange editors into multiple panes and view several files side by side or in a grid.

For example, four independent model canvases can be arranged in a two-by-two grid, allowing the user to inspect multiple models in parallel. Each canvas instance binds to a single model

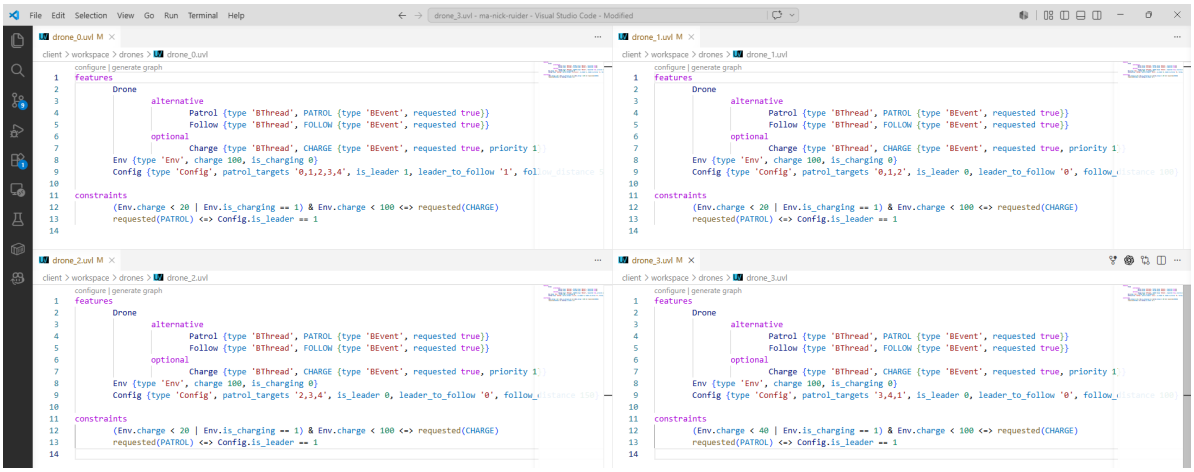


Figure 4.7: VS Code with four independent text editor instances.

file and maintains its own runtime overlay state for visualization. In Figure 4.7, four different `drones.uvl` files are displayed simultaneously using this feature. We use a similar setup for the graphical editor. To guarantee correct event routing in multi-model layouts, SSE messages must include the file path of the affected model. When the active view for that file is visible, transient updates like Env attribute updates and b-event highlighting are applied to that view only. This VS Code integration thus enables coordinated multi-model inspection while preserving provenance for BP-specific runtime updates. This multi-model arrangement refines RQ1 by keeping several visualized models inspectable at once.

5 Implementation

In this chapter, we present the technical realization of the graphical editor for managing UVL-based feature models with BP semantics. It describes the metamodel extension for BP, the GLSP setup, the FMBP runtime extension, and the synchronization layer between the new tool and the runtime environment.

5.1 ANTLR Parser Integration and Metamodel Extensions for BP

This section describes the concrete realization of the conceptual BP integration we introduce in Section 4.2. It covers the grammar extensions, parser adaptations, and metamodel classes that constitute the implementation.

5.1.1 ANTLR Grammar Extensions

The grammar modifications are confined to `UVLParser.g4` in the ANTLR grammar module of the UVL metamodel component. They realize the opt-in BP extension described at the concept level while leaving all remaining productions unchanged, thereby preserving backward compatibility. The extensions comprise three targeted additions, shown in Listing 5.1. First, we augment the `features` rule to allow up to two additional top-level features, `Env` and `Config`. Second, we extend the `constraint` rule with a new labeled alternative `bpConstraint`, which groups all BP-specific constraint types under a single production to maintain an organized grammar structure. Third, within `bpConstraint`, a dedicated `conflictingConstraint` production handles the multi-reference case of mutually conflicting b-events. To ensure deterministic lexing, we introduce a dedicated keyword token for each `bp` construct, as shown in Listing 5.2.

```
features: FEATURES_KEY NEWLINE INDENT feature envConfigFeature? envConfigFeature?
    DEDENT;

envConfigFeature:
   reference attributes? NEWLINE;

constraint:
   ...
   | bpConstraint # BPEventConstraint
   | ...;

bpConstraint:
   requestedConstraint
   | blockedConstraint
   | waitedForConstraint
   | selectedConstraint
   | conflictingConstraint;

conflictingConstraint:
   CONFLICTING_KEY OPEN_PAREN (reference COMMA)* reference CLOSE_PAREN;
```

Listing 5.1: ANTLR grammar extensions for BP integration.

```
REQUESTED_KEY: 'requested';
BLOCKED_KEY: 'blocked';
WAITED_FOR_KEY: 'waited_for';
SELECTED_KEY: 'selected';
CONFLICTING_KEY: 'conflicting';
```

Listing 5.2: ANTLR lexer extensions for BP integration.

5.1.2 Parser and Factory Adaptations

We adapt the parsing logic in two places: the ANTLR listener and the model factory. The `ModelType` enumeration controls model variant selection, introduced to support distinct parsing strategies under a single parser entry point, and may be extended to accommodate additional model variants in the future. When `ModelType.BP` is active, the `UVLListener` constructor instantiates a `BPFeatureModel`. Otherwise, it falls back to the standard `FeatureModel`, as shown in Listing 5.3.

ANTLR generates paired `enterX` and `exitX` callbacks for each grammar rule. A so-called `ParseTreeWalker` invokes `enterX` before visiting a node's children and `exitX` after all child callbacks have completed. The `enter` methods initialize objects and push them onto a stack, while `exit` methods finalize those objects using data populated by child callbacks [70].

The additional top-level feature rule illustrates this pattern concretely. Its grammar production is `reference attributes? NEWLINE`, meaning that when the walker enters the rule, the

```
public UVLListener(ModelType modelType) {
   FeatureModel featureModel = modelType == ModelType.BP
      ? new BPFeatureModel()
      : new FeatureModel();
   this.fmBuilder = new FeatureModelBuilder(featureModel);
}
```

Listing 5.3: `UVLListener` constructor with model type selection.

`reference` and `attributes` subtrees are accessible but not yet resolved. Both listener methods begin with a call to `rejectIfBaseModel`, which enforces the opt-in nature of BP parsing. When the active model type is not `ModelType.BP`, any BP-specific construct encountered in the listener is rejected with a parse error, and the method returns immediately.

Accordingly, `enterEnvConfigFeature` creates a `Feature` and pushes it onto the `featureStack`. Child listeners populate the feature instance from its child parse nodes, namely the `reference` and optional `attributes` subtrees. When we invoke `exitEnvConfigFeature`, the `Feature` is popped from the stack and assigned to either the `Env` or `Config` field. Any feature name other than these two gets rejected. Listing 5.4 shows the corresponding listener methods.

```
@Override
public void enterEnvConfigFeature(UVLJavaParser.EnvConfigFeatureContext ctx) {
   if (rejectIfBaseModel("Env/Config top-level feature", ctx.getStart().getLine())
       ) return;
   String featureName = ctx.reference().getText().replace("\"", "");
   Feature feature = new Feature(featureName);
   featureStack.push(feature);
}

@Override
public void exitEnvConfigFeature(UVLJavaParser.EnvConfigFeatureContext ctx) {
   if (rejectIfBaseModel("Env/Config top-level feature", ctx.getStart().getLine())
       ) return;
   Feature feature = featureStack.pop();
   BPFeatureModel featureModel = (BPFeatureModel) fmBuilder.getFeatureModel();
   if (feature.getFeatureName().equals("Env")) {
      featureModel.setEnv(feature);
   } else if (feature.getFeatureName().equals("Config")) {
      featureModel.setConfig(feature);
   } else {
      errorList.add(new ParseError(
         "Only features named Env and Config are allowed as additional top-level
             features!"));
   }
   featureModel.getFeatureMap().put(feature.getFeatureName(), feature);
}
```

Listing 5.4: Listener methods for BP top-level features (Env / Config).

BP constraints follow a similar enter/exit pattern. The enter method instantiates the appropriate constraint object, and the exit method finalizes it and registers it with the model. For `ConflictingConstraint`, which carries multiple references, the listener accumulates all reference objects into a list during child callbacks and passes that list to the constructor upon exit. The full implementations are available in the accompanying source code [77].

5.1.3 Metamodel Extensions

The metamodel classes introduced in this subsection are the same ones instantiated by the listener described in the previous Section 5.1.2. They provide the structural definitions that the listener populates during parsing.

The central addition is `BPFeatureModel`, which extends `FeatureModel` and introduces two dedicated fields for the `Env` and `Config` top-level features. Both are represented as standard `Feature` objects and are distinguished semantically by their type attribute definition. Listing 5.5 shows an excerpt of the class structure.

```
public class BPFeatureModel extends FeatureModel {
   private Feature env;
   private Feature config;

   public Feature getEnv() { return env; }
   public void setEnv(Feature env) { this.env = env; }

   public Feature getConfig() { return config; }
   public void setConfig(Feature config) { this.config = config; }

   ...
}
```

Listing 5.5: `BPFeatureModel` class with `Env` and `Config` top-level features.

Another example of a metamodel addition is the new `ConflictingConstraint`. Rather than holding a single reference, it manages an ordered list of `VariableReference` objects. The class is shown in Listing 5.6.

All remaining BP-specific constraint additions follow the same structural pattern: a grammar production, a listener that constructs the constraint object, and registration with the model upon exit. Single-reference BP constraints extend `AbstractBPEventConstraint`, which is a shared base class that encapsulates the common single-event reference structure. Further implementation details are available in the source code [77].

In summary, we realize the BP integration through targeted extensions at each layer. All changes are strictly additive with respect to the standard UVL pipeline, yielding a BP-capable

```java
public class ConflictingConstraint extends Constraint {
   private final List<VariableReference> references;

   public ConflictingConstraint(List<VariableReference> references) { ... }

   public List<VariableReference> getReferenceList() {
      return Collections.unmodifiableList(references);
   }
   public void setReference(int index, VariableReference reference) {
      this.references.set(index, reference);
   }
   @Override
   public String toString(boolean withSubmodels, String currentAlias) { ... }
}
```

Listing 5.6: `ConflictingConstraint` class for multi-reference BP constraints (excerpt).

parsing infrastructure that is backward-compatible with existing UVL models and structurally aligned with the conceptual architecture developed in this thesis.

5.2 GLSP Implementation Overview

This section provides a structured overview of the GLSP implementation, covering both the server and the client components. First, we describe the server, as it owns the domain model, the GLSP session state, and the operation handlers that implement editing behavior. The client is presented afterward, with a focus on rendering logic and the VS Code integration. We include some representative code excerpts to illustrate key points, but the full implementation is available in the accompanying code repository for readers who want to explore further [77].

```java
public static void main(final String[] args) {
   new UVLServerLauncher().launch(args);
}

protected ServerModule createServerModule() {
   return new ServerModule().configureDiagramModule(createDiagramModule());
}

protected DiagramModule createDiagramModule() {
   return new UVLDiagramModule();
}
```

Listing 5.7: Server launcher and module bootstrap.

5.2.1 Server bootstrap

The server entry point is `UVLServerLauncher`, which parses command-line arguments, instantiates the diagram module, and starts the socket-based GLSP server. The launcher is intentionally minimal, as shown in Listing 5.7, delegating all configuration logic to the module system.

`UVLDiagramModule` serves as the server's central configuration point. It registers all services relevant to the GLSP server with the dependency injection framework. Listing 5.8 shows examples for source storage, model state, and handlers. The full module is available in the source code repository [77].

```
public class UVLDiagramModule extends DiagramModule {
   @Override
   protected Class<? extends SourceModelStorage> bindSourceModelStorage() {
      return UVLSourceModelStorage.class;
   }

   @Override
   protected Class<? extends UVLModelState> bindGModelState() {
      return UVLModelStateImpl.class;
   }

   @Override
   protected void configureOperationHandlers(final MultiBinding<OperationHandler
       <?>> binding) {
      binding.add(UVLApplyLabelEditOperationHandler.class);
      binding.add(UVLDeleteOperationHandler.class);
      binding.add(UVLCreateFeatureOperationHandler.class);
      binding.add(UVLCreateAttributeOperationHandler.class);
      binding.add(UVLCreateRelationEdgeOperationHandler.class);
      ...
   }
   ...
}
```

Listing 5.8: Essential bindings in the `UVLDiagramModule`.

Registered operation handlers provide a concrete example of how UVL-specific editing logic is integrated into the GLSP runtime. Each handler translates an incoming GLSP operation into a targeted mutation of both the UVL domain model and its GModel representation. Handlers corresponding to generic actions, such as label editing and deletion, subclass existing GLSP framework classes and augment them to update the UVL feature model. For example, `UVLApplyLabelEditOperationHandler` extends the already existing framework class `GModelApplyLabelEditOperationHandler` by adding a step that propagates the name change to the domain model. Handlers for UVL-specific operations, such as creating features, attributes, and relation edges, implement the handler interface directly.

5.2.2 Source model management and session state

As discussed in Section 4.1, a UVL file should contain only data relevant to the feature tree or the behavioral program. Therefore, the paired-file design keeps the two concerns strictly separate. The server manages, loads, and saves both files while also keeping the in-memory domain model and notation model synchronized throughout editing sessions.

We manage persistence by using the `UVLSourceModelStorage` class, which maintains a paired representation of each diagram. On load, it reads the textual `.uvl` file into an in-memory feature model and simultaneously reads the adjacent notation file, which shares the same base name but carries the extension `.notation.json`. The notation file encodes the GLSP graph state in JSON, including element identifiers and layout coordinates. Both files are loaded and saved together, ensuring that we preserve the visual state across editing sessions. For the initial load, if the notation file is missing, the server initializes an empty GModel based on the feature model. Listing 5.9 shows the method responsible for loading the paired files.

```
public void loadSourceModel(RequestModelAction action) {
   File featureModelFile = convertToFile(action.getOptions());
   validateModelPath(featureModelFile, ClientOptionsUtil.getSourceUri(action.
       getOptions()));
   loadFeatureModel(featureModelFile);

   File notationFile = getNotationFile(featureModelFile.getAbsolutePath());
   loadGModel(notationFile);
}
```

Listing 5.9: Excerpt of `UVLSourceModelStorage` showing the loading of paired source and notation files.

```
public interface UVLModelState extends GModelState {
   FeatureModel getFeatureModel();

   void setFeatureModel(FeatureModel model);

   void updateIndex();

   @Override
   UVLModelIndex getIndex();
}
```

Listing 5.10: `UVLModelState` based on the `GModelState`.

The second core element is `UVLModelState`, which acts as the main injectable entry point for the active domain model and its notation data. GLSP injects this model state into many other components, so centralizing the session data here keeps handlers and helpers focused on their actual responsibilities. The interface builds on the existing GLSP `GModelState` abstraction and adds direct access to the feature model as well as an explicit hook for updating the

derived index. In this way, the model state combines the generic GLSP session structure with the thesis-specific UVL domain model. Listing 5.10 shows the extended interface.

The implementation details of `UVLModelStateImpl` are intentionally omitted from the thesis because they primarily constitute framework infrastructure. Conceptually, the state keeps the current feature model and notation model together and updates the derived index whenever the domain model changes. The description is sufficient for understanding the architecture, while the concrete mechanics can be inspected in the source code repository [77].

GModel elements use string identifiers, whereas the UVL domain model is a structured object graph. This mismatch requires a dedicated mapping from diagram element IDs to their corresponding domain objects and vice versa. We implement the mapping in `UVLModelIndex`, a session-local helper owned by the model state and also defined in Listing 5.10. The index is updated whenever the domain model changes, ensuring that any handler can reliably resolve an incoming element ID to its corresponding domain object.

5.2.3 Extending `uvl.glsp` to `uvl.bp.glsp`

```
public class BPDiagramModule extends UVLDiagramModule {
   @Override
   protected Class<? extends SourceModelStorage> bindSourceModelStorage() {
      return BPSourceModelStorage.class;
   }

   @Override
   protected Class<? extends BPModelState> bindGModelState() {
      return BPModelStateImpl.class;
   }

   @Override
   protected void configureOperationHandlers(final MultiBinding<OperationHandler
       <?>> binding) {
      super.configureOperationHandlers(binding);

      binding.rebind(UVLApplyLabelEditOperationHandler.class,
          BPApplyLabelEditOperationHandler.class);
      binding.rebind(UVLDeleteOperationHandler.class, BPDeleteOperationHandler.
          class);
      binding.add(BPCreateBThreadOperationHandler.class);
      binding.add(BPCreateEventOperationHandler.class);
   }
   ...
}
```

Listing 5.11: `BPDiagramModule` overriding the existing bindings where necessary.

In Section 4.1.4, the package structure was designed to separate the core UVL functionality from the BP-specific extensions. This separation is reflected in the implementation, extending the base `uvl.glsp` package with a `uvl.bp.glsp` package. The `BPDiagramModule` subclasses `UVLDiagramModule`, overriding injected elements where necessary while reusing the common bindings. Listing 5.11 shows how the BP variant replaces the source model storage and the model state implementation while updating the operation handlers.

In this package, `BPSourceModelStorage` applies the parsing logic using the BP grammar variant, while `BPModelStateImpl` can extend `UVLModelStateImpl` to use the `BPFeatureModel` metamodel class. Additionally, we override the operation handlers to implement BP-specific editing logic and add new operation handlers responsible for creating b-threads and b-events. The replacements are localized to the module class and require no invasive changes to the GLSP framework or to existing handlers.

5.2.4 Client rendering packages

The `uvl-sprotty` package contains most of the rendering logic for the GLSP client. The central file, `uvl-diagram-module.ts`, binds all UVL-specific user interface behavior into the runtime. Listing 5.12 shows an excerpt of this module.

```
const uvlDiagramModule = new ContainerModule((bind, unbind, isBound, rebind) => {
   const context = {bind, unbind, isBound, rebind};
   ...
   bindAsService(context, TYPES.IAnchorComputer, CenteredAnchor);
   ...
   bind(HighlightElementsActionHandler).toSelf().inSingletonScope();
   configureActionHandler(context, HighlightElementAction.KIND,
       HighlightElementsActionHandler);

   configureDefaultModelElements(context);
   ...
   configureModelElement(context, UVLModelTypes.FEATURE, FeatureNode,
       FeatureNodeView);
   configureModelElement(context, UVLModelTypes.ATTRIBUTE, EditableGCompartment,
       GCompartmentView);
   configureModelElement(context, UVLModelTypes.REQUIRES, GEdge,
       SingleArrowEdgeView);
   ...
});
```

Listing 5.12: Excerpt from `uvl-diagram-module.ts` showing the registered UVL elements.

Comparing this listing with the server-side `UVLDiagramModule` shown earlier reveals an intentional structural symmetry. The GLSP framework applies an inversion-of-control pattern based on dependency injection (DI) to both sides of the client–server boundary. On the server, Google Guice manages the bindings [78]. On the client side, InversifyJS implements

the same pattern [79]. In both cases, we register every service and component in a global DI container.

This flexibility is expressed concretely through the element registrations in the client module. Each call to `configureModelElement` associates a UVL type constant with a model class and a view class. When the container receives a model element of type `UVLModelTypes.FEATURE`, for instance, it instantiates a custom `FeatureNode` as the model object and delegates rendering to `FeatureNodeView`, which produces the corresponding SVG output. The same pattern applies uniformly to attributes, relation edges, and other model elements.

Beyond element registration, the module also wires interaction and layout behavior into the container. For example, the `CenteredAnchor` overrides the default anchor computation to center edge connections between parent and child features. Highlight actions are received by the `HighlightElementsActionHandler` and then applied by the graphical view. Although the handler is implemented generically and could be reused by other server components, in this implementation it is primarily used to highlight b-events selected in the behavioral program view.

Similarly, the `uvl-bp-sprotty` package creates a separate container module that contains the BP-specific UI elements and behavior. We present the composition of these client-side modules in the VS Code integration in the following Section 5.3 on multi-profile packaging.

5.3 GLSP-based multi-profile VS Code extensions

This section shows how we derive multiple VS Code extension variants from one GLSP-based client codebase, as introduced in Section 4.1.4. The goal is to produce profile-specific products without duplicating repositories or requiring branch-heavy maintenance.

5.3.1 Manifest-driven profile definitions

We declare the profiles introduced by this work in a manifest used by the packaging scripts. Each entry pairs an extension identity with a server JAR, container modules, and command contribution modules. The VS Code extension assembles these artifacts during the build, and publishes the resulting `.vsix` as a profile-specific product. The split between container modules and command contributions is necessary because we inject them at different stages of the GLSP VS Code integration. In Listing 5.13, we show the `uvl` profile definition. The `uvl-bp` profile contains additional BP-specific modules.

This manifest-driven approach centralizes metadata, making it easier to manage and review. It also decouples profile definitions from build logic, improving maintainability and reducing

```
{
  "defaultProfile": "uvl",
  "profiles": [
      {
         "id": "uvl",
         "outFile": "uvl-vscode.vsix",
         "serverJarPath": "../../server/uvl.glsp/target/uvl.glsp-0.5.1-glsp.jar",
         "containerModuleIds": ["uvl-core"],
         "commandContributionIds": ["uvl-default"]
      },
      ...
   ]
}
```

Listing 5.13: Excerpt from `profiles.json` with `uvl` profile.

```
{
   "command": "uvl.sse.startListening",
   "title": "Start listening for FMBP",
   "enablement": "uvl.buildProfile == 'uvl-bp' && activeCustomEditorId == 'uvl.
      glspDiagram'"
}
```

Listing 5.14: Command contribution in `package.json` with profile gating.

the risk of divergence across products. Now that the profiles are defined, the next step is to implement the build and packaging logic to produce the corresponding VS Code extensions.

The `package.json` manifest defines a VS Code extension, declaring contribution points such as commands, custom editors, keybindings, and menus [80]. Rather than duplicating that manifest and the VS Code integration code across variants, the implemented approach uses a single extension with conditional contributions based on the selected profile. The *webpack* configuration resolves the selected profile at build time, injects compile-time constants, and copies the matching server JAR into the extension `dist` directory. Each VSIX therefore contains exactly one server artifact.

Context keys such as `uvl.buildProfile` gate profile-specific behavior in the `package.json` manifest. For example, the command contributions for the SSE listener are only included in the manifest when we select the `uvl-bp` profile due to the additional `enablement` condition on the command contribution. Listing 5.14 shows a sample command.

This approach maintains a single manifest while selectively exposing UI elements per profile. In practice, `buildProfile` is the final guardrail, preventing a contribution from becoming visible when it belongs to another profile.

5.3.2 Build-time injection

Command- and container-module wiring is driven from the profile manifest. The packaging step reads the entries in `profiles.json` and collects the identifiers of the client-side container modules and the command contribution modules for each profile. During build, those identifiers are emitted into a small registry that the VS Code extension uses at startup to assemble profile-specific functionality.

The GLSP VS Code integration expects an extension-level starter class to return the primary DI container for the diagram client. In this implementation, we subclass this starter so the container can be composed from the core 'uvl-sprotty' modules plus any additional plugin modules selected for the active profile. Instead of returning a single monolithic container, the starter resolves additional plugin modules from the registry generated by `profiles.json` and supplies them during container initialization. Listing 5.15 shows the 'UvlDiagramStarter' used in the VS Code integration as an example. This setup lets the same starter code produce different runtime compositions without duplicating the client wiring.

```
class UvlDiagramStarter extends GLSPStarter {
   createContainer(...containerConfiguration: ContainerConfiguration): Container {
      const pluginModules = resolveWebviewPluginModules();
      return initializeUvlDiagramContainer(new Container(), ...pluginModules, ...
          containerConfiguration);
   }
}
```

Listing 5.15: `UvlDiagramStarter` class that composes the main container from core and profile-provided plugin modules.

We handle command contributions in a similar, manifest-driven way, but they are registered at a different integration point. Further details about the implementation of the registries and the build-time injection of the command contributions are available in the source code and are not reproduced here for brevity [77].

5.4 Extending FMBP

In this section, we implement the conceptually defined extension of FMBP described in Section 4.3. The additional components for SSE transport are the `EventStreamServer` and `EventStreamFactory`. `ContextSourceStreamDecorator` and `BEventStreamerExtension` are responsible for event production.

5.4.1 SSE Transport

To send runtime information to the GLSP server, we extend FMBP with a transport layer based on SSE. Events are published over a single SSE stream exposed by a Flask application that runs on a background daemon thread [81]. The `EventStreamServer` class encapsulates the Flask application, the event queue, and the background thread, as shown in Listing 5.16. Clients probe the service using `/health` and consume the stream using `/events`.

```
class EventStreamServer:
   def __init__(self, port: int = 8099) -> None:
      self.__port = port
      self.__app = Flask("fmbp-event-stream")
      self.__events: Queue[str] = Queue(maxsize=10000)
      self.__started = False
      self.__flask_task = Thread(
         target=self.__app.run,
         kwargs={"port": self.__port, "threaded": True},
         daemon=True,
      )
      self.__app.route("/events")(self.__events_endpoint)
      self.__app.route("/health")(self.__health_endpoint)
```

Listing 5.16: Constructor of `EventStreamServer`.

The `/events` handler waits on the event queue with a fifteen-second timeout. On timeout, the handler yields a keep-alive comment in order to prevent proxies from closing the connection. Listing 5.17 shows this implementation.

```
def __events_endpoint(self) -> Response:
   @stream_with_context
   def generate():
      yield ": connected\n\n"
      while True:
         try:
            payload = self.__events.get(timeout=15)
            yield _format_sse(payload)
         except Empty:
            yield ": keep-alive\n\n"
   response = Response(generate(), mimetype="text/event-stream")
   ...
   return response
```

Listing 5.17: SSE endpoint with keep-alive handling.

The `publish` method is the sole entry point for producers. The method enforces a bounded queue by dropping the oldest entry when the queue is full. Each payload must include a `source` key that clients can use to differentiate events from multiple publishers. Figure 5.18 shows an excerpt of this method.

```python
def publish(self, payload: dict[str, Any]) -> None:
   ...
   serialized = json.dumps(payload, default=str)
   try:
      self.__events.put_nowait(serialized)
   except Full:
      try:
         self.__events.get_nowait()
         self.__events.put_nowait(serialized)
      except Empty:
         pass
```

Listing 5.18: Queue-based `publish` method.

5.4.2 Extension Components

As described in Section 4.3.3, we implement two complementary extension components to produce the runtime information for the GLSP server. The context source decorator sends environment snapshots, while the configurator extension sends b-event selection details.

Context Source Decorator

```python
class ContextSourceStreamDecorator(ContextSource):
   def __init__(
         self,
         source: str,
         context_source: ContextSource,
         event_stream_server: EventStreamServer,
   ) -> None:
      self.__source = source
      self.__context_source = context_source
      self.__event_stream_server = event_stream_server

   def get_data(self) -> CONTEXT_DATA:
      data = self.__context_source.get_data()
      self.__event_stream_server.publish({
         "timestamp": datetime.now(timezone.utc).isoformat(),
         "source": self.__source,
         "type": "context_update",
         "data": data,
      })
      return data
```

Listing 5.19: `ContextSourceStreamDecorator` for publishing context snapshots.

The `ContextSourceStreamDecorator` wraps a `ContextSource`, applying the *Decorator* design pattern [73]. On each call to `get_data()`, it publishes a `context_update` message and

returns the original result unchanged to the caller. Listing 5.19 presents the straightforward implementation.

Additionally, the decorator performs deduplication by comparing the current snapshot with the previously published one and emitting an event only if they differ, as shown in the project's source code [77]. The original context source is unaware of the decorator and requires no modification, making this extension fully opt-in and non-invasive.

B-Event Streamer Extension

We extend `BPConfigurator` through the `BPConfiguratorExtension` protocol to enable b-event streaming. This protocol mirrors the listener hooks that `BPConfigurator` already uses during execution. The protocol definition and the exposed hooks are shown in Listing 5.20.

```
class BPConfiguratorExtension(Protocol):
    def starting(self, b_program: BProgram) -> None:
        ...

    def event_selected(self, b_program: BProgram,
                       event: BEvent) -> None:
        ...
```

Listing 5.20: `BPConfigurator` extension protocol with listener hooks.

We pass the extensions to the `BPConfigurator` at construction time. During execution, the configurator invokes all registered extensions with failure isolation, so that a single faulty extension cannot interrupt the runtime. Listing 5.21 shows the constructor-based integration.

```
class BPConfigurator(SimpleBProgramRunnerListener):
    def __init__(
            ...,
            extensions: Iterable[BPConfiguratorExtension] | None = None,
    ) -> None:
        ...
        self.__extensions = tuple(extensions or ())
```

Listing 5.21: `BPConfigurator` constructor accepting extensions.

`BEventStreamerExtension` is a concrete implementation of `BPConfiguratorExtension`. It initializes the SSE server in `starting` and handles selected events in `event_selected`. For each selected event, it derives the relation to each ticket, transforms the data into a JSON payload, and publishes the payload as an SSE message. This mapping and publication logic is implemented in Listing 5.22.

```python
class BEventStreamerExtension(BPConfiguratorExtension):
    ...
    def starting(self, b_program: BProgram) -> None:
        self.__event_stream_server.start()

    def event_selected(self, b_program: BProgram, event: BEvent) -> None:
        ...
        for ticket in b_program.tickets:
            request: BEvent | None = ticket.get("request")
            block: BEvent | None = ticket.get("block")
            wait_for: BEvent | None = ticket.get("waitFor")
            thread_name: str = b_program.get_name(ticket["bt"])

            if request and request.name == event.name:
                self.__publish_event("requested", thread_name, event.name)
            if block and block.name == event.name:
                self.__publish_event("blocked", thread_name, event.name)
            if wait_for and wait_for.name == event.name:
                self.__publish_event("waited_for", thread_name, event.name)

    def __publish_event(self, kind: str, thread_name: str, event_name: str) -> None:

        self.__event_stream_server.publish({
            "timestamp": datetime.now(timezone.utc).isoformat(),
            "source": self.__source,
            "type": "event_selected",
            "kind": kind,
            "thread": thread_name,
            "event": event_name,
        })
```

Listing 5.22: `BEventStreamerExtension` publishing event selection data.

5.4.3 EventStreamFactory

The `EventStreamFactory` serves as the public wiring API. The factory returns a context source decorator or a b-event stream extension configured with a shared SSE server.

As shown in Listing 5.23, `EventStreamFactory` binds all generated decorators and extensions to one shared `EventStreamServer` instance. This keeps concurrent runtime models on a single SSE endpoint, while the `source` parameter preserves model-specific routing on the GLSP side. To add streaming to a scenario, we create a factory and request the decorator and the extension for the appropriate `source` path.

```python
class EventStreamFactory:
    def __init__(self, port: int = 8099) -> None:
        self.__server = EventStreamServer(port=port)

    def b_event_stream(self, source: str) -> BEventStreamerExtension:
        return BEventStreamerExtension(source, self.__server)

    def context_source(
            self, source: str, context_source: ContextSource
    ) -> ContextSourceStreamDecorator:
        return ContextSourceStreamDecorator(source, context_source, self.__server)
```

Listing 5.23: `EventStreamFactory` for shared streaming setup.

5.4.4 Example Usage

In the following examples, we show how the introduced extensions to FMBP can be used. The water tank example covers the basic wiring pattern. The drones example shows how the same pattern scales to multiple concurrent instances that share a single stream.

Water Tank

The water tank scenario publishes context updates, such as water level and temperature, along with b-event selections. A dedicated `ContextSource` reads from the shared tank state, as shown in Listing 5.24.

```python
class WaterTankContextSource(ContextSource):
    def get_data(self) -> dict[str, str | int | float | bool]:
        return {
            "temp": TANK.water_temperature,
            "level": TANK.water_level,
        }
```

Listing 5.24: Context source reading the shared tank state.

In the first step, we keep the configuration provider setup. The `WaterTankContextSource()` is replaced by a wrapped instance from the factory. The call to `context_source(...)` on the factory returns a decorated source that publishes snapshots while the provider chain remains unchanged. Listing 5.25 shows the new instantiation. We pass the UVL file path as the source value so that emitted context updates can be mapped to the corresponding editor.

In the second step, we pass the preconfigured instances to the behavioral program. The `BPConfigurator` injects the object through its `extensions` parameter. The method call

```
event_stream_factory = EventStreamFactory(port=8099)

config_provider = CachingConfigurationProvider(
   ContextConfigurationProvider(
      event_stream_factory.context_source(
         source=str(uvl_path.resolve()),
         context_source=WaterTankContextSource(),
      ),
      interface,
   ),
)
```

Listing 5.25: Configuration provider with wrapped `WaterTankContextSource()`.

`b_event_stream(...)` receives the UVL file path as the source value. Listing 5.26 shows the extended `FMBProgram`.

From this point onward, the endpoint at `http://localhost:8099/events` delivers a single aggregated stream that carries both context update and b-event selection messages. Each message is tagged with the UVL file path that identifies its source.

```
b_program = FMBProgram(
   bthreads=[add_hot(), add_cold(), remove_water(), finished()],
   event_selection_strategy=PriorityBasedEventSelectionStrategy(),
   listener=BPConfigurator(
      WaterTankListener(TANK),
      config_provider,
      DynamicConsistencyChecker(interface),
      MTimeUpdatingModelWatcher(interface),
      extensions=[event_stream_factory.b_event_stream(
         source=str(uvl_path.resolve()),
      )],
   ),
)
b_program.run()
```

Listing 5.26: Program instantiation with injected streaming extension.

Drones

The drones scenario runs four instances concurrently. We create the factory once and reuse it for multiple instances. Each one wraps its own `DroneContextSource` and registers its own `BEventStreamerExtension` while targeting the shared server, as underlined in Listing 5.27.

Each drone publishes events with a distinct source value that equals its UVL file path. The single aggregated stream at `/events` therefore remains separable by clients even when multiple publishers are active.

```
event_stream_factory = EventStreamFactory(port=8099)

for i in range(4):
   uvl_path = workspace_path / "drones" / f"drone_{i}.uvl"

   config_provider = CachingConfigurationProvider(
      ContextConfigurationProvider(
         event_stream_factory.context_source(
            source=str(uvl_path.resolve()),
            context_source=DroneContextSource(),
         ),
         interface,
      ),
   )

   b_program = FMBProgram(
      ...
      listener=BPConfigurator(
         DroneListener(8000 + i, queue),
         ...
         extensions=[event_stream_factory.b_event_stream(
            source=str(uvl_path.resolve()),
         )],
      ),
   )
   process = DroneProcess(b_program)
   process.start()
```

Listing 5.27: Four drones sharing one `EventStreamFactory`.

Taken together, these examples demonstrate that we can extend the existing FMBP structure without major changes to the framework. The streaming components and extension hooks transmit runtime data to the GLSP server, enabling the graphical UVL editor to implement the *models@run.time* approach. Further integration details involving the receiving side of the stream in the form of the GLSP server appear in Section 5.5.

5.5 *Models@run.time* Integration in GLSP

This section describes how we integrate *models@run.time* into the GLSP-based BP variant. The VS Code extension contributes only the user-facing commands that start and stop the listener. The server owns the SSE connection and filters the events it receives that are relevant to the current session. Once active, the server translates the received messages into GLSP actions and dispatches them to the client, which updates the diagram accordingly.

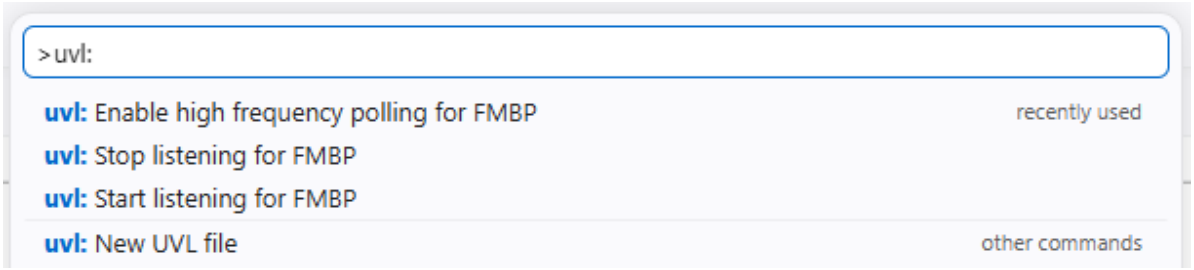


Figure 5.1: Command palette for UVL-BP in VS Code.

5.5.1 Initiating the connection from VS Code

We manage the SSE connection via three VS Code commands contributed by the BP build profile. They are gated in the extension manifest, so they appear only when the BP-profile custom editor is active. The `start listening` command calls the server to attempt to establish the connection using a polling strategy that starts at a low frequency. The `enable high-frequency polling` command switches to a more aggressive polling mode when needed. We can also close the connection when it is no longer required. All three commands are available from the default command palette in VS Code. Invoking any command dispatches the corresponding GLSP action through the diagram connector. Listing 5.28 shows an example of how the command contribution is defined in the codebase, while Figure 5.1 displays the resulting command palette in VS Code.

```
export function configureSSECommandContributions(context: CommandContext): void {
   const { extensionContext, diagramPrefix, connector } = context;

   extensionContext.subscriptions.push(
      vscode.commands.registerCommand(`${diagramPrefix}.sse.startListening`, () =>
          {
         connector.dispatchAction(SSEStartListeningAction.create());
      }),
      ...
   );
}
```

Listing 5.28: Excerpt of SSE command contributions in the VS Code extension.

5.5.2 Server-Side Wiring

In Section 5.2.3, we introduce the `BPDiagramModule` as the central wiring point for the UVL-BP server. In this class, we override the `configureAdditionals` method to register the SSE-related services as singletons. Listing 5.29 shows the setup of the services. The`FMBPEventListenerService` maintains the connection to the external FMBP event stream.

`FMBPHighlightActionDispatchService` and `FMBPContextUpdateService` independently consume parsed events and translate them into the appropriate GLSP action dispatches.

`FMBPEventListenerService` implements the network layer. On startup, the service waits for the FMBP health endpoint to become reachable before opening the event stream at `/events`. A dedicated background thread consumes the stream, and the service automatically reconnects if the connection is lost. Listing 5.29 shows the implementation.

We parse each raw payload into a `ParsedServerSentEvent`, a normalized record that exposes the event type, source path, timestamp, and structured data map. This representation abstracts over the raw JSON lines and allows downstream services to filter and route events without coupling them to the wire format. Crucially, the `source` field identifies the feature model from which an event originates, allowing services to discard events that do not belong to the currently open diagram.

```
@Override
protected void configureAdditionals() {
   super.configureAdditionals();

   bind(ServerSentEventsService.class).to(FMBPEventListenerService.class).
       asEagerSingleton();
   bind(FMBPHighlightActionDispatchService.class).asEagerSingleton();
   bind(FMBPContextUpdateService.class).asEagerSingleton();
}
```

Listing 5.29: Singleton bindings for SSE integration in `BPDiagramModule`.

5.5.3 Translating SSE into GLSP Actions

The listener does not manipulate the diagram directly. Instead, services register as data listeners on the `ServerSentEventsService` and are invoked selectively for the SSE types they handle. This publisher-subscriber design keeps each service focused on a single concern and makes it straightforward to add support for new event types without modifying existing code.

Visual feedback via highlight actions. `FMBPHighlightActionDispatchService` subscribes to `requested`, `blocked`, and `waited_for` events. When such an event arrives, the service resolves the corresponding GLSP element identifiers from the b-thread and b-event name embedded in the payload. The service creates a `HighlightElementAction` if a matching element exists, and then dispatches it to the client, as seen in Listing 5.30.

The `HighlightElementAction` constitutes the primary visual feedback loop of the *models@run.time* integration. The client receives these actions and highlights the corresponding

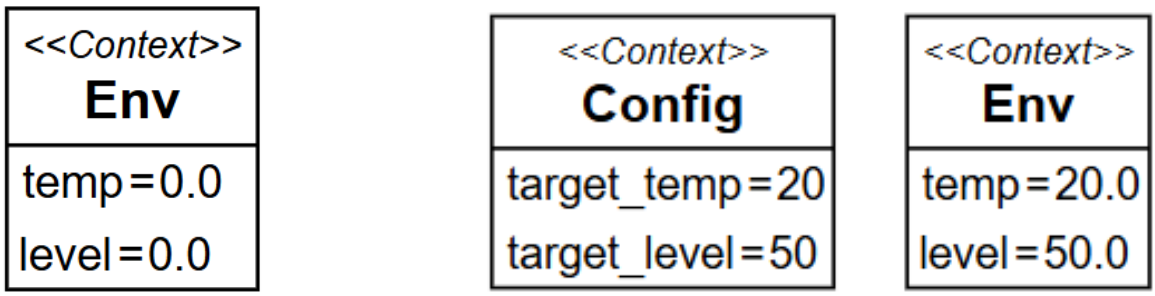
```
@Inject
protected void registerDataListener() {
   serverSentEventsService.addDataListener(this::dispatchHighlightAction, "
       requested", "blocked", "waited_for");
}

protected void dispatchHighlightAction(final ParsedServerSentEvent event) {
   Optional<String> threadName = parseThreadName(event);
   Optional<String> eventName = parseEventName(event);

   Set<String> matchingFeatureIds = eventName.isPresent()
      ? resolveFeatureIdsByEventName(threadName.get(), eventName.get())
      : resolveFeatureIdsByThreadName(threadName.get());

   actionDispatcher.dispatch(
      new HighlightElementAction(List.copyOf(matchingFeatureIds), true));
}
```

Listing 5.30: `FMBPHighlightActionDispatchService` dispatches highlight action for selected b-events.

Figure 5.2: Highlighted b-thread of water tank example.

elements, providing an immediate visual indication of the active execution state without any polling. The visual feedback is transient and rendered solely via CSS on the client side. Figure 5.2 displays the result, which is oriented at the concept in Section 4.4.6.

Env synchronization via context updates. `FMBPContextUpdateService` subscribes exclusively to `context_update` events. Upon receipt, the service compares the incoming attribute values with those in the current model state, applies any changes, rebuilds the affected graphical node, and resubmits the updated model. Listing 5.31 shows an excerpt of the implementation. The client thereby receives a refreshed diagram representation. At this stage, the changes to the `Env` feature are visible in the diagram, but not persisted in the underlying model. This is a deliberate design choice to keep the implementation focused on runtime visualization and to avoid invoking a save action from the server. We can still manually save the model to persist the changes if desired. Figure 5.3 displays an updated context due to the execution of a behavioral program.



(a) Before execution of BP (b) Updated Env in Finished state

Figure 5.3: Env feature context update in water tank example.

```
@Inject
protected void registerDataListener() {
   serverSentEventsService.addDataListener(this::updateContextEnv,
       CONTEXT_UPDATE_TYPE);
}

protected void updateContextEnv(final ParsedServerSentEvent event) {
   Feature envFeature = modelState.getFeatureModel().getEnv();
   ...
   for (Map.Entry<String, ?> entry : event.data().entrySet()) {
      if (envFeature.getAttributes().containsKey(entry.getKey())) {
         Attribute attribute = envFeature.getAttributes().get(entry.getKey());
         if (!Objects.equals(attribute.getValue(), entry.getValue())) {
            attribute.setValue(entry.getValue());
            hasChanges = true;
         }
      }
   }

   if (hasChanges) actionDispatcher.dispatchAll(submitModel(envFeature));
}
```

Listing 5.31: Incremental context update and model resubmission in `FMBPContextUpdateService`.

The presented excerpts provide only a brief insight into the overall implementation of the tool. Many additional components, configurations, and integration details necessarily fall outside the scope of this thesis. Readers interested in low-level implementation details, concrete class structures, or the full extent of the framework integrations are encouraged to consult the complete source code, which is publicly available on GitHub and can be freely inspected [77, 82].

6 Evaluation

In the evaluation chapter, we demonstrate the practical application of the proposed toolchain through concrete example scenarios. The examples illustrate how the editor can be used to model feature trees, integrate behavioral programming concepts, and synchronize with runtime models. It concludes with a summary of the overall results of this thesis.

6.1 Examples

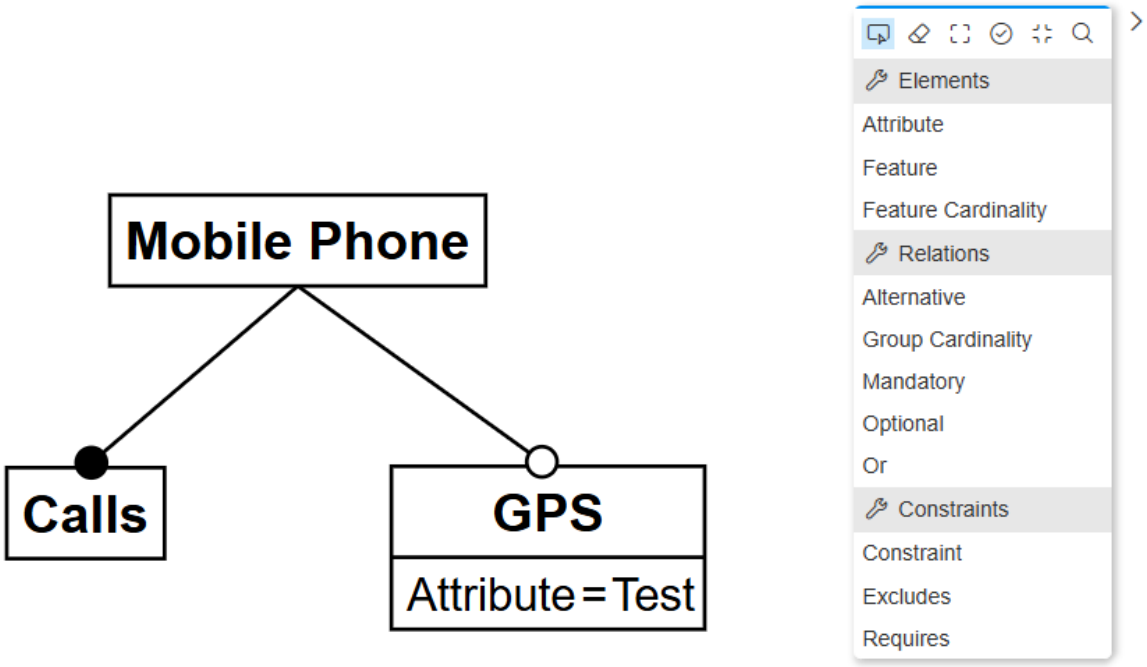
The following examples revisit the three domains used throughout the thesis, each highlighting a different aspect of the toolchain. Taken together, we show how the proposed VS Code extensions behave in practice.

6.1.1 Mobile Phone

As a first example, we address the mobile phone example shown in Figure 2.2 and recreate it in the base UVL GLSP editor. It provides a compact but representative baseline for checking whether the tool can model feature trees as intended. The example is particularly useful because it combines the core editing operations with a familiar structure that can be verified against the textual UVL representation in Listing 2.1.

We start a new UVL model using the command contribution and create a root feature using the tool palette on the right, as shown in Figure 6.1b. New features are then added by selecting the feature element in the palette and clicking the intended parent feature. In this workflow, the editor appends a child feature to the root and, by default, assigns an *optional* relation. To change the relation type between two elements, we select the desired relation type, such as *mandatory*, and then click on the parent and child features. The editor updates the parent-child relationship immediately and stores this change in the UVL file upon saving.

A feature name can be changed by double-clicking the corresponding node. A text box overlay appears, allowing the user to enter a new name. The editor then resizes the feature element automatically so that the new label still fits. The same interaction pattern is also available for other labels, including feature and group cardinalities. Although the mobile phone example does not include attributes, the editor supports them as well via the tool palette and by clicking the target feature. Figure 6.1a shows a sample of the described actions.



(a) Feature nodes with relations and an attribute (b) Tool palette

Figure 6.1: Working with the UVL GLSP extension.

When a feature node has exactly one child of a given relation type, newly appended children inherit that relation as the default value. This is useful for repeated *alternative* and *or* structures, because it reduces repetitive editing.

We define complex constraints directly in the constraint box, provided they form a valid expression for the current feature model. For the commonly used *requires* and *excludes* constraints, a more convenient alternative is available. They can be added directly via the tool palette and are rendered as cross-tree constraints within the tree, with routing points to preserve layout clarity.

The capabilities of the UVL-GLSP VS Code extension described above allow us to recreate the full mobile phone example. This can be achieved either by building the model from scratch or by opening the existing UVL file from Listing 2.1 in the editor Through changes to the positions of the nodes, the feature tree of Figure 2.2 can be recreated, as seen in Figure 6.2.

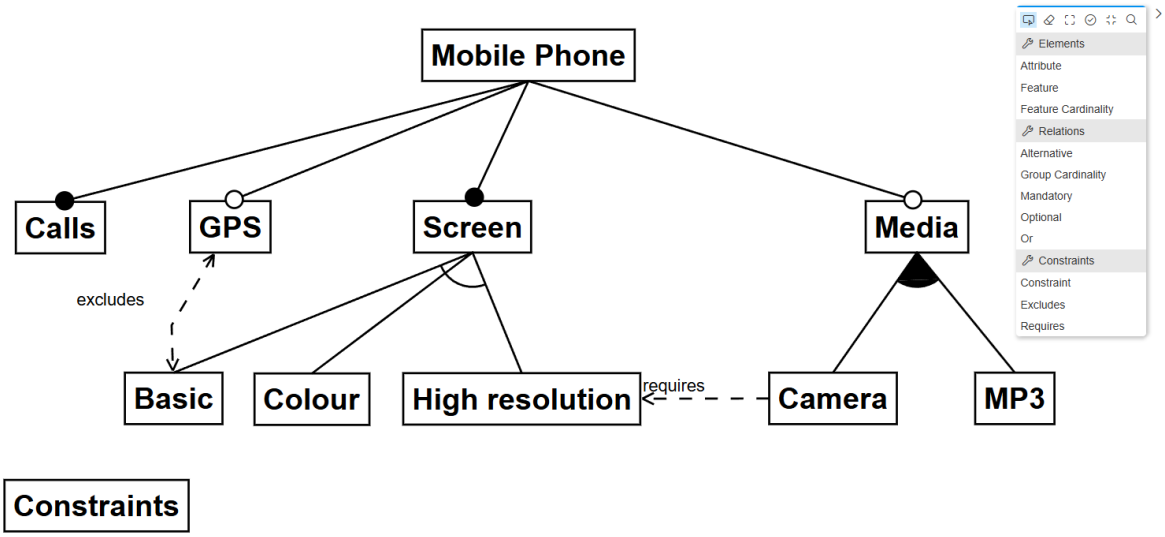


Figure 6.2: Mobile Phone example modeled in VS Code with UVL GLSP extension.

6.1.2 Water Tank

The water tank example shifts the focus from structural editing to behavioral programming and *models@run.time*. First, we show how the editor can add BP-specific elements to the feature model to recreate the water tank example of Section 2.1.3. Then, using FMBP as the framework for running the behavioral program, the UVL-BP VS Code extension demonstrates how the editor can receive updates from the running system and keep the model synchronized in the graphical view.

Integration of BP Elements

To model the water tank example described in Section 2.1.3, we deactivate the base UVL extension and enable the UVL-BP VS Code extension. Listing 6.1 shows the UVL file used to represent the water tank example [4, 40].

When copying the UVL file into the editor's workspace and opening it, the extension renders the feature tree including the new *b-thread* and *b-event* elements. Additionally, the editor displays both context features, `Config` and `Env`, next to the default constraint list. The visual representation of these BP-specific elements is aligned with the concept introduced in Section 4.4. If the water tank example is extended with additional b-threads and b-events, the user can use the new elements in the tool palette to add them to the model. A b-thread can be added by selecting the palette element and clicking on the intended parent feature. B-events can be added similarly to attributes, by selecting the tool palette element and clicking on the target feature. This creates a sub-compartment for attributes and b-events. The water tank example is visually represented in the editor in Figure 6.3. Updating the BP-specific elements

```
features
   Watertank
      [1..4]
         AddHot {type 'BThread',
               HOT {type 'BEvent', requested true, priority 1}}
         AddCold {type 'BThread',
                COLD {type 'BEvent', requested true, priority 1}}
         RemoveWater {type 'BThread',
                   DRAIN {type 'BEvent', requested true, priority 2}}
         Finished {type 'BThread',
                 FINISHED {type 'BEvent', requested true, priority 1}}
   Env {type 'Env', temp 0, level 0}
   Config {type 'Config', target_temp 20, target_level 50}

constraints
   conflicting(HOT, COLD, DRAIN, FINISHED)
   (Env.temp < Config.target_temp) => requested(HOT)
   (Env.temp > Config.target_temp) => requested(COLD)
   (Env.level > Config.target_level) <=> selected(DRAIN)
   (Env.temp == Config.target_temp & Env.level == Config.target_level) <=>
       selected(FINISHED)
```

Listing 6.1: UVL specification of the water tank feature model. Taken from [40].

is done through the same interactions as the base UVL elements, such as double-clicking a b-thread to change its name or changing the value of a `Config` variable.

Runtime Synchronization

In Section 5.4.4, we extend FMBP to support *models@run.time* in the GLSP editor. We use the same setup here to run the water tank example. The implementation details are available in the source code [82].

We open the UVL file with the BP VS Code extension and start the high-polling command for a short period. This makes the extension poll the FMBP SSE endpoint at an increased frequency until the behavioral program is started. Once the program is running, the GLSP server connects to it and listens for incoming events. The server parses the received events and forwards the corresponding actions to the GLSP client. As a result, the `Env` feature updates continuously, and the b-threads are highlighted based on the behavioral program's current state. Figure 6.4 illustrates this behavior, showing both the highlighted b-thread and the changed `Env` context feature.

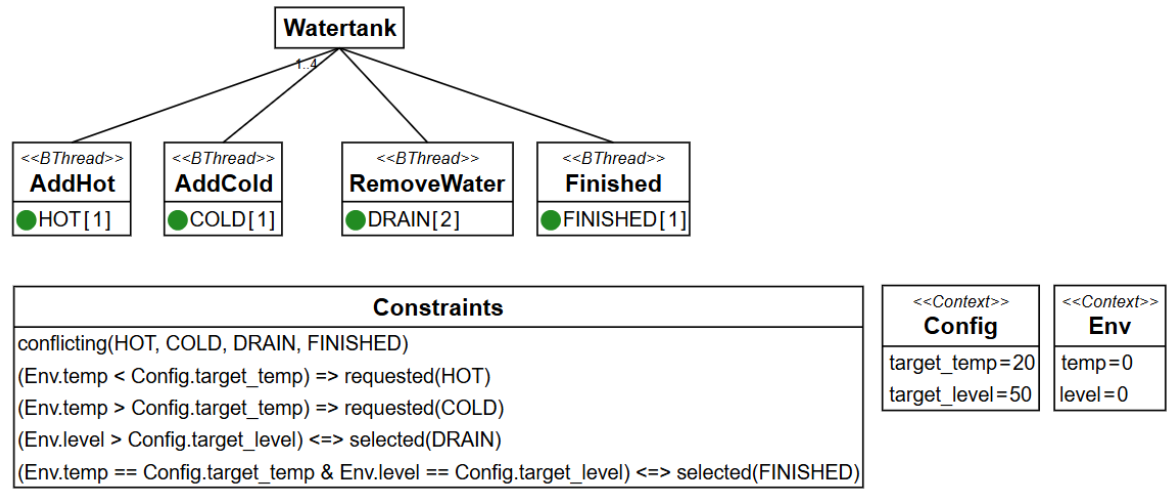


Figure 6.3: Water Tank example modeled in VS Code with UVL-BP GLSP extension.

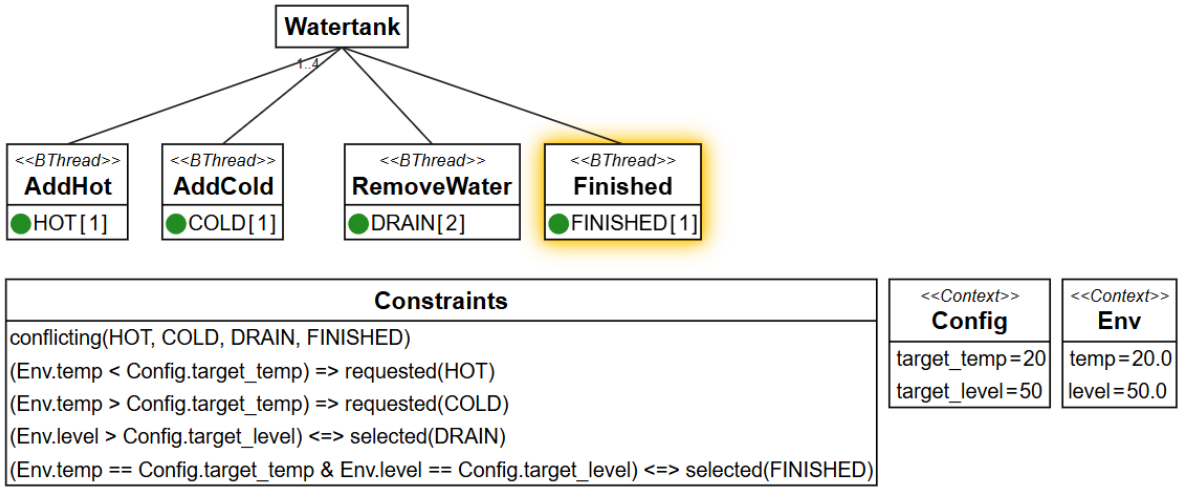


Figure 6.4: Water Tank example showing runtime synchronization; compare to Figure 6.3.

6.1.3 Drones

An essential example for evaluating this thesis is the use of the proposed tool across multiple concurrent runtime models. For this purpose, we use the drones example from FMBP [4] to demonstrate how the editor can handle several concurrent instances of a behavioral program.

Setup of the Drones Example

Similar to the water tank example, we implement the behavioral program for the drones in FMBP. Section 5.4.4 describes a snapshot of the implementation. The setup of the drones

example requires running the behavioral program in FMBP and initializing the *Alchemist* simulation environment to visualize the drones' movements. The instructions for the setup are available in the source code and won't be addressed here for brevity [82].

In addition to setting up the behavioral program, we use the UVL-BP VS Code extension to open the UVL files for the drones example. In this case, we open four separate UVL files, each representing the feature model of one drone. To gain insights into the concurrent execution of these drones, we arrange the editor windows in a two-by-two grid, allowing all four models to be visible simultaneously. Each model includes three b-threads, each with one b-event, as well as the context feature `Env`, which displays the drone's current energy level. The editor allows the user to change each drone's settings via the `Config` feature and the constraints. Section 2.6 provides more details on the configuration. In this demonstration, drone 0 is configured as the leader, while the other three drones are set as followers. Also, drone 3 is configured with a low energy threshold to trigger charging behavior early.

Demonstration in VS Code

To establish the connection between the running behavioral program and the editor, we start the high-polling command in the VS Code client. This call initiates the connection to FMBP SSE and starts receiving updates from all four drones. As described in Section 5.4.4, the SSE are mapped through the respective UVL file path. Initially, the views display the selected b-events for each drone. Drone 0 shows the active b-event for the *Patrol* b-thread, while drones 1 to 3 show the active b-event for the *Follow* b-thread. As the simulation progresses, the energy level of drone 3 drops below the configured threshold, which triggers the *Charge* b-thread and highlights the corresponding b-event in the editor. In Figure 6.5, the state of all four drones is shown in the editor, with drone 3 highlighted for charging and the other three drones highlighted for their respective roles. Additionally, the energy levels of all drones are visible in the `Env` feature.

During the evaluation of the drones example, we identified several performance limitations. The concurrent execution of FMBP, *Alchemist*, and four GLSP instances demands substantial computational resources, resulting in degraded responsiveness and unstable rendering. It is important to emphasize that the developed tool serves as a proof of concept, demonstrating the technical feasibility of visualizing a running behavioral program in real time. However, to achieve production-level reliability, we require further optimization of both the GLSP implementation and the backend infrastructure.

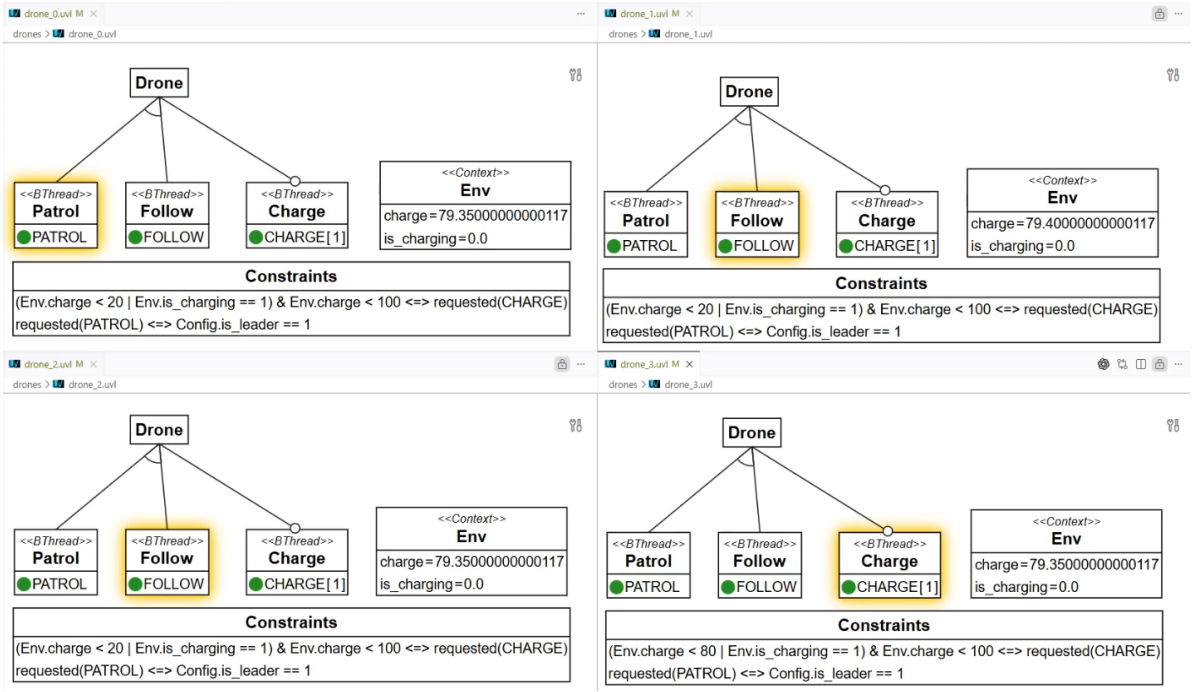


Figure 6.5: Drones example demonstration showing the runtime synchronization.

6.2 Results

The results of this thesis can be summarized along three main contributions. We develop two VS Code extensions based on the GLSP framework. The base extension supports UVL feature modeling and provides a graphical editor for creating, updating, and serializing feature trees. The second extension builds on the same foundation and adds BP-specific elements, allowing behavioral programming concepts to be represented directly within the same editor. The thesis extends Felber's FMBP implementation with runtime integration capabilities. This addition enables the editor to exchange runtime information with a running behavioral program and makes the graphical view reactive to the current execution state. Although we use this integration within the UVL-BP extension, the implementation is structured to support other environments in the future.

The examples demonstrate that these contributions work together as a coherent toolchain. The mobile phone example shows that the base editor can model feature trees, capture change relations, update labels, and keep the textual UVL representation synchronized with the graphical state. Afterward, the water tank example shows how the same environment can represent BP-specific elements and synchronize the editor with runtime updates from the behavioral program. Finally, the drones example extends this setup to multiple concurrent instances, showing that the approach remains usable when several models are active simultaneously.

At the same time, the implementation should be viewed as a proof of concept rather than a

finished product. Several design choices are guided by established GLSP examples, such as the workflow example and BigUML, mentioned in Section 3.3, as they provide proven interaction patterns for editor development. Those choices are reasonable for a first implementation, but we can still refine them to better suit the specific needs of the UVL environment. In addition, some capabilities already available in FeatureIDE for UVL are not yet implemented. Submodels are one important example. The three examples presented do not require them, yet they would be valuable for other UVL models and future UVL-BP scenarios.

Performance remains another limitation of the runtime integration. The current solution is adequate for the water tank and, in part, for the drones examples, but larger models may cause noticeable delays during synchronization. The lack of performance is already irregularly visible in the presented scenarios. For that reason, further work should focus on reducing overhead in the runtime communication path and on making the editor more robust for larger models.

Overall, we show that a GLSP-based UVL editor can be extended into a combined modeling and runtime environment. We also show that the same architecture can integrate feature modeling and behavioral programming into a single workflow. The resulting system is sufficient to demonstrate the concept and support the three examples, while leaving ample room for future improvements in completeness, scalability, and runtime performance.

7 Conclusion

In this thesis, we introduce a graphical editor based on the GLSP framework for UVL feature models and extend it to support BP-enriched UVL feature models. The toolchain aims to enhance the comprehension of contextual behavioral programs across multiple runtimes. In this chapter, we summarize the main contributions of this thesis with regard to the research questions. Additionally, we outline potential avenues for future work, which could further enhance the capabilities and performance of the developed graphical editor for UVL feature models and its integration with behavioral programming concepts.

7.1 Contributions

We present a graphical editor for UVL feature models, implemented on top of the GLSP framework. The editor supports BP-enriched UVL feature models and synchronizes model changes with running behavioral programs. The implementation includes VS Code extensions that enable users to author and edit UVL feature models in the editor. The toolchain supports *models@run.time* for behavioral programs and allows runtime modifications of feature models to be reflected in the running system. The repository publishes two separate extensions on *OpenVSX* [83] with one extension targeting base UVL models [84] and the other targeting UVL models enriched with BP annotations [85].

The contributions of this thesis are relevant to the research questions posed in Section 1.2.

RQ1: The editor demonstrates the feasibility of graphical visualization for UVL feature models by providing an interactive visual representation built on GLSP. The editor's design allows users to visually explore the structure and relationships of feature models. To improve comprehension of contextual BP, we provide visual cues and annotations that help users understand how features relate to behavioral programming concepts.

RQ2: We integrate BP concepts into an extensible graphical tool using the GLSP framework. The editor's architecture is designed to separate the core UVL modeling concerns from the BP extensions, enabling future enhancements. It uses a profile-based approach to package two separate VS Code extensions.

RQ3.1: We extend FMBP to provide runtime information about the executing behavioral program using SSE. The GLSP server consumes the events and updates the graphical model accordingly. This includes transient modifications to the `Env` attributes and highlights to b-threads when certain b-events are selected in the behavioral program.

RQ3.2 We use the GLSP editor in combination with Felber's FMBP to run behavioral programs. Because they operate on the same underlying UVL model, user interactions that modify the feature model in the editor can be directly translated into changes in the running behavioral program. Primarily, we can modify the constraints and `Config` attributes of the feature model, thereby altering the runtime behavior of the executed behavioral program.

7.2 Future Work

Performance and Scalability. Further engineering effort is required to improve the editor's performance for larger and concurrent feature models. This includes implementing incremental update strategies for the most expensive operations and introducing selective environment modifications that avoid full model refreshes, reducing synchronization overhead and improving perceived responsiveness.

Architecture and Maintainability. The current UVL metamodel and parser would benefit from a refactor to separate grammar concerns from metamodel concerns. A clearer separation would ease maintenance and enable more modular parsing approaches for future language features. On the client side, smaller package modularization, similar to BigUML [35], combined with a broader client refactor would simplify reuse and enable deeper integration with the VS Code platform.

Visualization and Interaction. The visual presentation could be enhanced by adopting advanced interaction techniques that improve scalability. Semantic zooming and off-screen representations for less relevant details are promising directions for reducing visual clutter and supporting focus-plus-context interactions [86]. Interaction feedback should also be strengthened by adding element highlighting and tighter selection feedback; for example, selecting a complex constraint should highlight related features and attributes. The existing infrastructure already supports highlight dispatching and only requires the corresponding actions and style rules to be completed.

Editor Views and Usability. Additional editor views would make the tool more flexible for different tasks. For example, a compact tree view without attributes and a dedicated property panel for constraints would help users who prefer structured or textual representations.

Testing and Evaluation. Automated unit and end-to-end tests implemented with Playwright [87] would increase reliability and enable early detection of regressions. A user study should also be conducted to assess the editor both as a modeling tool and as a means to improve comprehension of behavioral programming concepts.

Bibliography

[1] David Harel, Assaf Marron, and Gera Weiss. “Behavioral programming”. In: *Commun. ACM* 55.7 (July 1, 2012), pp. 90–100. ISSN: 0001-0782. DOI: 10.1145/2209249.2209270.

[2] Tom Felber and Sebastian Götz. “Debugging Behavioral Programs Using Models@run.time”. In: *2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. 2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA). ISSN: 2376-9521. Aug. 2024, pp. 160–167. DOI: 10.1109/SEAA64295.2024.00032.

[3] Tom Felber and Sebastian Götz. “Behavioral Programming in the Large using Variability Management”. In: *Proceedings of the 19th International Working Conference on Variability Modelling of Software-Intensive Systems*. VaMoS ’25. New York, NY, USA: Association for Computing Machinery, May 28, 2025, pp. 126–130. ISBN: 979-8-4007-1441-2. DOI: 10.1145/3715340.3715443.

[4] Tom Felber and Sebastian Götz. “Comprehensible Self-Aware Behavioral Programming”. In: *2025 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*. 2025 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C). Sept. 2025, pp. 215–220. DOI: 10.1109/ACSOS-C66519.2025.00064.

[5] Dominik Bork, Philip Langer, and Tobias Ortmayr. “A Vision for Flexible GLSP-Based Web Modeling Tools”. In: *The Practice of Enterprise Modeling*. Ed. by João Paulo A. Almeida et al. Cham: Springer Nature Switzerland, 2024, pp. 109–124. ISBN: 978-3-031-48583-1. DOI: 10.1007/978-3-031-48583-1_7.

[6] Chico Sundermann et al. “Integration of UVL in FeatureIDE”. In: *Proceedings of the 25th ACM International Systems and Software Product Line Conference - Volume B*. SPLC ’21. New York, NY, USA: Association for Computing Machinery, Sept. 6, 2021, pp. 73–79. ISBN: 978-1-4503-8470-4. DOI: 10.1145/3461002.3473940.

[7] Gordon Blair, Nelly Bencomo, and Robert B. France. “Models@ run.time”. In: *Computer* 42.10 (Oct. 2009), pp. 22–27. ISSN: 1558-0814. DOI: 10.1109/MC.2009.326.

[8] Achiya Elyasaf. "Context-Oriented Behavioral Programming". In: *Information and Software Technology* 133 (May 1, 2021). ISSN: 0950-5849. DOI: 10.1016/j.infsof.2020.106504.

[9] Adiel Ashrov et al. "A Study on the Comprehensibility of Behavioral Programming Variants:" in: *Proceedings of the 20th International Conference on Evaluation of Novel Approaches to Software Engineering*. Porto, Portugal: SCITEPRESS - Science and Technology Publications, 2025, pp. 252–267. ISBN: 978-989-758-742-9. DOI: 10.5220/0013440800003928.

[10] Paul Clements and Linda Northrop. *Software Product Lines: Practices and Patterns*. Addison-Wesley, 2002. 563 pp. ISBN: 978-0-201-70332-0.

[11] Kwanwoo Lee, Kyo C. Kang, and Jaejoon Lee. "Concepts and Guidelines of Feature Modeling for Product Line Software Engineering". In: *Software Reuse: Methods, Techniques, and Tools*. Ed. by Cristina Gacek. Berlin, Heidelberg: Springer, 2002, pp. 62–77. ISBN: 978-3-540-46020-6. DOI: 10.1007/3-540-46020-9_5.

[12] Klaus Pohl, Günter Böckle, and Frank Van Der Linden. *Software Product Line Engineering*. Berlin, Heidelberg: Springer, 2005. ISBN: 978-3-540-28901-2. DOI: 10.1007/3-540-28901-1.

[13] S. Buhne, K. Lauenroth, and K. Pohl. "Modelling requirements variability across product lines". In: *13th IEEE International Conference on Requirements Engineering (RE'05)*. Aug. 2005, pp. 41–50. DOI: 10.1109/RE.2005.45.

[14] Sven Apel and Christian Kästner. "An Overview of Feature-Oriented Software Development." In: *The Journal of Object Technology* 8.5 (2009). ISSN: 1660-1769. DOI: 10.5381/jot.2009.8.5.c5.

[15] Johannes Bürdek et al. "Staged configuration of dynamic software product lines with complex binding time constraints". In: *Proceedings of the 8th International Workshop on Variability Modelling of Software-Intensive Systems*. VaMoS '14. New York, NY, USA: Association for Computing Machinery, Jan. 22, 2014, pp. 1–8. ISBN: 978-1-4503-2556-1. DOI: 10.1145/2556624.2556627.

[16] Rafael Capilla et al. "An overview of Dynamic Software Product Line architectures and techniques: Observations from research and industry". In: *Journal of Systems and Software* 91 (May 1, 2014), pp. 3–23. ISSN: 0164-1212. DOI: 10.1016/j.jss.2013.12.038.

[17] Kyo C Kang et al. "Feature-oriented domain analysis (FODA) feasibility study". In: (1990).

[18] Krzysztof Czarnecki, Simon Helsen, and Ulrich Eisenecker. "Formalizing cardinality-based feature models and their specialization". In: *Software Process: Improvement and Practice* 10.1 (2005), pp. 7–29. ISSN: 1099-1670. DOI: 10.1002/spip.213.

[19] Krzysztof Czarnecki, Simon Helsen, and Ulrich Eisenecker. "Staged Configuration Using Feature Models". In: *Software Product Lines*. Ed. by Robert L. Nord. Berlin, Heidelberg: Springer, 2004, pp. 266–283. ISBN: 978-3-540-28630-1. DOI: 10.1007/978-3-540-28630-1_17.

[20] David Benavides, Pablo Trinidad, and Antonio Ruiz-Cortés. "Automated Reasoning on Feature Models". In: *Advanced Information Systems Engineering*. Ed. by Oscar Pastor and João Falcão e Cunha. Berlin, Heidelberg: Springer, 2005, pp. 491–503. ISBN: 978-3-540-32127-9. DOI: 10.1007/11431855_34.

[21] David Benavides, Sergio Segura, and Antonio Ruiz-Cortés. "Automated analysis of feature models 20 years later: A literature review". In: *Information Systems* 35.6 (Sept. 1, 2010), pp. 615–636. ISSN: 0306-4379. DOI: 10.1016/j.is.2010.01.001.

[22] José A. Galindo et al. "Automated analysis of feature models: Quo vadis?" In: *Computing* 101.5 (May 1, 2019), pp. 387–433. ISSN: 1436-5057. DOI: 10.1007/s00607-018-0646-1.

[23] Clément Quinton, Daniel Romero, and Laurence Duchien. "Cardinality-based feature models with constraints: a pragmatic approach". In: *Proceedings of the 17th International Software Product Line Conference*. SPLC '13. New York, NY, USA: Association for Computing Machinery, Aug. 26, 2013, pp. 162–166. ISBN: 978-1-4503-1968-3. DOI: 10.1145/2491627.2491638.

[24] Ahmet Serkan Karataş and Halit Oğuztüzün. "Attribute-based variability in feature models". In: *Requirements Engineering* 21.2 (June 1, 2016), pp. 185–208. ISSN: 1432-010X. DOI: 10.1007/s00766-014-0216-9.

[25] Chico Sundermann et al. "Yet another textual variability language? a community effort towards a unified language". In: *Proceedings of the 25th ACM International Systems and Software Product Line Conference - Volume A*. SPLC '21. New York, NY, USA: Association for Computing Machinery, Sept. 6, 2021, pp. 136–147. ISBN: 978-1-4503-8469-8. DOI: 10.1145/3461001.3471145.

[26] Nelly Bencomo, Sebastian Götz, and Hui Song. "Models@run.time: a guided tour of the state of the art and research challenges". In: *Software & Systems Modeling* 18.5 (Oct. 1, 2019), pp. 3049–3082. ISSN: 1619-1374. DOI: 10.1007/s10270-018-00712-x.

[27] René Schöne et al. "Incremental Runtime-generation of Optimisation Problems using RAG-controlled Rewriting". In: *Proceedings of the 11th International Workshop on Models@run.time Co-located with 19th International Conference on Model Driven Engineering Languages and Systems (MODELS 2016)* (2016), pp. 26–34.

[28] Hendrik Bünder. "Decoupling Language and Editor - The Impact of the Language Server Protocol on Textual Domain-Specific Languages:" in: *Proceedings of the 7th International Conference on Model-Driven Engineering and Software Development*. 7th International Conference on Model-Driven Engineering and Software Development. Prague, Czech Republic: SCITEPRESS - Science and Technology Publications, 2019, pp. 129–140. ISBN: 978-989-758-358-2. DOI: 10.5220/0007556301290140.

[29] Roberto Rodriguez-Echeverria et al. “Towards a Language Server Protocol Infrastructure for Graphical Modeling”. In: *Proceedings of the 21th ACM/IEEE International Conference on Model Driven Engineering Languages and Systems*. MODELS ’18. New York, NY, USA: Association for Computing Machinery, Oct. 14, 2018, pp. 370–380. ISBN: 978-1-4503-4949-9. DOI: 10.1145/3239372.3239383.

[30] *eclipse-glsp/glsp-server*. URL: https://github.com/eclipse-glsp/glsp-server (visited on 05/04/2026).

[31] *eclipse-glsp/glsp-server-node*. URL: https://github.com/eclipse-glsp/glsp-server-node (visited on 05/04/2026).

[32] *eclipse-glsp/glsp-client*. URL: https://github.com/eclipse-glsp/glsp-client (visited on 05/04/2026).

[33] *eclipse-glsp/glsp-vscode-integration*. URL: https://github.com/eclipse-glsp/glsp-vscode-integration (visited on 05/04/2026).

[34] *eclipse-glsp/glsp-theia-integration*. URL: https://github.com/eclipse-glsp/glsp-theia-integration (visited on 05/04/2026).

[35] Haydar Metin and Dominik Bork. “On Developing and Operating GLSP-based Web Modeling Tools: Lessons Learned from BIGUML”. In: *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. 2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS). Oct. 2023, pp. 129–139. DOI: 10.1109/MODELS58315.2023.00031.

[36] *eclipse-glsp/glsp-examples*. URL: https://github.com/eclipse-glsp/glsp-examples (visited on 05/04/2026).

[37] Jill H. Larkin and Herbert A. Simon. “Why a Diagram is (Sometimes) Worth Ten Thousand Words”. In: *Cognitive Science* 11.1 (Jan. 1, 1987), pp. 65–100. ISSN: 0364-0213. DOI: 10.1016/S0364-0213(87)80026-5.

[38] Daniel Moody. “The “Physics” of Notations: Toward a Scientific Basis for Constructing Visual Notations in Software Engineering”. In: *IEEE Transactions on Software Engineering* 35.6 (Nov. 2009), pp. 756–779. ISSN: 1939-3520. DOI: 10.1109/TSE.2009.67.

[39] John Sweller. “Cognitive load theory and educational technology”. In: *Educational Technology Research and Development* 68.1 (2020), pp. 1–16. ISSN: 1042-1629.

[40] Tom Felber. *tfelbr/FMBP*. URL: https://github.com/tfelbr/FMBP (visited on 05/27/2026).

[41] D Pianini, S Montagna, and M Viroli. “Chemical-oriented simulation of computational systems with ALCHEMIST”. In: *Journal of Simulation* 7.3 (Aug. 1, 2013), pp. 202–215. ISSN: 1747-7778. DOI: 10.1057/jos.2012.27.

[42] David Benavides et al. “UVL: Feature modelling with the Universal Variability Language”. In: *Journal of Systems and Software* 225 (July 1, 2025). ISSN: 0164-1212. DOI: 10.1016/j.jss.2024.112326.

[43] Jacob Loth et al. “UVLS: A Language Server Protocol For UVL”. In: *Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume B*. Vol. B. SPLC ’23. New York, NY, USA: Association for Computing Machinery, Aug. 28, 2023, pp. 43–46. ISBN: 979-8-4007-0092-7. DOI: 10.1145/3579028.3609014.

[44] *Universal-Variability-Language/uvl-lsp*. URL: https://github.com/Universal-Variability-Language/uvl-lsp (visited on 05/07/2026).

[45] *Z3Prover/z3*. URL: https://github.com/Z3Prover/z3 (visited on 05/07/2026).

[46] *Graphviz*. Graphviz. URL: https://graphviz.org/ (visited on 05/07/2026).

[47] Christian Kastner et al. “FeatureIDE: A tool framework for feature-oriented software development”. In: *2009 IEEE 31st International Conference on Software Engineering*. 2009 IEEE 31st International Conference on Software Engineering. ISSN: 1558-1225. May 2009, pp. 611–614. DOI: 10.1109/ICSE.2009.5070568.

[48] *FeatureIDE/FeatureIDE*. URL: https://github.com/FeatureIDE/FeatureIDE (visited on 05/07/2026).

[49] Jens Meinicke et al. *Mastering Software Variability with FeatureIDE*. Cham: Springer International Publishing, 2017. ISBN: 978-3-319-61443-4. DOI: 10.1007/978-3-319-61443-4.

[50] Francisco Sebastian Benitez et al. “UVL web-based editing and analysis with flamapy.ide”. In: *Proceedings of the 19th International Working Conference on Variability Modelling of Software-Intensive Systems*. VaMoS ’25. New York, NY, USA: Association for Computing Machinery, May 28, 2025, pp. 121–125. ISBN: 979-8-4007-1441-2. DOI: 10.1145/3715340.3715436.

[51] *flamapy/flamapy-ide*. URL: https://github.com/flamapy/flamapy-ide (visited on 05/07/2026).

[52] José A. Galindo et al. “FLAMA: A collaborative effort to build a new framework for the automated analysis of feature models”. In: *Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume B*. Vol. B. SPLC ’23. New York, NY, USA: Association for Computing Machinery, Aug. 28, 2023, pp. 16–19. ISBN: 979-8-4007-0092-7. DOI: 10.1145/3579028.3609008.

[53] *flamapy/flamapy*. URL: https://github.com/flamapy/flamapy (visited on 05/07/2026).

[54] *pyodide/pyodide*. URL: https://github.com/pyodide/pyodide (visited on 05/07/2026).

[55] *d3/d3*. URL: https://github.com/d3/d3 (visited on 05/07/2026).

[56] Nir Eitan et al. “On Visualization and Comprehension of Scenario-Based Programs”. In: *2011 IEEE 19th International Conference on Program Comprehension*. 2011 IEEE 19th International Conference on Program Comprehension. ISSN: 1092-8138. June 2011, pp. 189–192. DOI: 10.1109/ICPC.2011.10.

[57] David Harel, Assaf Marron, and Gera Weiss. *Programming Coordinated Behavior in Java*. Dec. 25, 2010. 250 pp. ISBN: 978-3-642-14106-5. DOI: 10.1007/978-3-642-14107-2_12.

[58] Tom Felber. *tfelbr/uvl-bp-lsp*. URL: https://github.com/tfelbr/uvl-bp-lsp (visited on 05/26/2026).

[59] *eclipse-sprotty/sprotty*. URL: https://github.com/eclipse-sprotty/sprotty (visited on 05/09/2026).

[60] *eclipse-glsp/glsp-eclipse-integration*. URL: https://github.com/eclipse-glsp/glsp-eclipse-integration (visited on 05/09/2026).

[61] *eclipse-glsp/glsp*. URL: https://github.com/eclipse-glsp/glsp (visited on 05/09/2026).

[62] Markus Hegedüs. "Real-time collaborative modeling with eclipse GLSP". Thesis. Technische Universität Wien, 2023. DOI: 10.34726/hss.2023.102183.

[63] Jonas Helming, Maximilian Koegel, and Philip Langer. *Real-time Collaboration on Diagrams with Eclipse GLSP*. EclipseSource. URL: https://eclipsesource.com/blogs/2024/02/21/real-time-collaboration-on-diagrams-with-eclipse-glsp/ (visited on 05/09/2026).

[64] *Universal-Variability-Language/java-fm-metamodel*. URL: https://github.com/Universal-Variability-Language/java-fm-metamodel (visited on 05/15/2026).

[65] Chico Sundermann et al. "UVLParser: Extending UVL with Language Levels and Conversion Strategies". In: *Proceedings of the 27th ACM International Systems and Software Product Line Conference - Volume B*. Vol. B. SPLC '23. New York, NY, USA: Association for Computing Machinery, Aug. 28, 2023, pp. 39–42. ISBN: 979-8-4007-0092-7. DOI: 10.1145/3579028.3609013.

[66] *Technology | 2025 Stack Overflow Developer Survey*. Stack Overflow. URL: https://survey.stackoverflow.co/2025/technology#1-dev-id-es (visited on 05/26/2026).

[67] Tina Schrepfer. *What is a Visual Studio VSIX package file?* Visual Studio. URL: https://learn.microsoft.com/en-us/visualstudio/extensibility/anatomy-of-a-vsix-package?view=visualstudio (visited on 05/26/2026).

[68] *Universal-Variability-Language/uvl-parser*. URL: https://github.com/Universal-Variability-Language/uvl-parser (visited on 05/15/2026).

[69] *antlr/antlr4*. URL: https://github.com/antlr/antlr4 (visited on 05/15/2026).

[70] Terence Parr. "The Definitive ANTLR 4 Reference". In: (2013). (Visited on 05/15/2026).

[71] Luis de la Torre et al. "Using Server-Sent Events for Event-Based Control in Networked Control Systems". In: *IFAC-PapersOnLine* 52 (Jan. 1, 2019), pp. 260–265. DOI: 10.1016/j.ifacol.2019.08.218.

[72] Ramírez, Sebastián. *Server-Sent Events (SSE) - FastAPI*. FastAPI. URL: https://fastapi.tiangolo.com/tutorial/server-sent-events/ (visited on 05/15/2026).

[73] Erich Gamma et al. "Design Patterns: Abstraction and Reuse of Object-Oriented Design". In: *ECOOP' 93 — Object-Oriented Programming*. Ed. by Oscar M. Nierstrasz. Berlin, Heidelberg: Springer, 1993, pp. 406–431. ISBN: 978-3-540-47910-9. DOI: 10.1007/3-540-47910-4_21.

[74] Tom Yaacov. "BPpy: Behavioral programming in Python". In: *SoftwareX* 24 (Dec. 1, 2023). ISSN: 2352-7110. DOI: 10.1016/j.softx.2023.101556.

[75] Bossert, Andre. *Add support for "attributed feature models" · Issue #255 · FeatureIDE/FeatureIDE*. GitHub. June 9, 2018. URL: https://github.com/FeatureIDE/FeatureIDE/issues/255#issuecomment-395977132 (visited on 05/15/2026).

[76] Microsoft. *Default keyboard shortcuts reference*. Visual Studio Code Documentation. URL: https://code.visualstudio.com/docs/reference/default-keybindings (visited on 05/27/2026).

[77] Nick Ruider. *xXNicksdaXx/ma-nick-ruider*. URL: https://github.com/xXNicksdaXx/ma-nick-ruider (visited on 05/26/2026).

[78] *google/guice*. URL: https://github.com/google/guice (visited on 05/25/2026).

[79] *inversify/monorepo*. URL: https://github.com/inversify/monorepo (visited on 05/25/2026).

[80] Microsoft. *Extension Manifest*. Visual Studio Code. URL: https://code.visualstudio.com/api/references/extension-manifest (visited on 05/26/2026).

[81] *pallets/flask*. URL: https://github.com/pallets/flask (visited on 05/27/2026).

[82] Nick Ruider. *xXNicksdaXx/FMBP-SSE*. URL: https://github.com/xXNicksdaXx/FMBP-SSE (visited on 05/28/2026).

[83] Sven Efftinge and Miro Spönemann. *Eclipse Open VSX: A Free Marketplace for VS Code Extensions | The Eclipse Foundation*. 2020. URL: https://www.eclipse.org/community/eclipse_newsletter/2020/march/1.php (visited on 05/30/2026).

[84] Nick Ruider. *UVL VS Code Extension*. Open VSX Registry. URL: https://open-vsx.org/extension/NickRuider/uvl-vscode-extension (visited on 05/30/2026).

[85] Nick Ruider. *UVL VS Code Extension (BP)*. Open VSX Registry. URL: https://open-vsx.org/extension/NickRuider/uvl-bp-vscode-extension (visited on 05/30/2026).

[86] Giuliano De Carlo, Philip Langer, and Dominik Bork. "Advanced visualization and interaction in GLSP-based web modeling: realizing semantic zoom and off-screen elements". In: *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems*. MODELS '22: ACM/IEEE 25th International Conference on Model Driven Engineering Languages and Systems. Montreal Quebec Canada: ACM, Oct. 23, 2022, pp. 221–231. ISBN: 978-1-4503-9466-6. DOI: 10.1145/3550355.3552412.

[87] *eclipse-glsp/glsp-playwright*. URL: https://github.com/eclipse-glsp/glsp-playwright (visited on 06/03/2026).